

Mars 2026

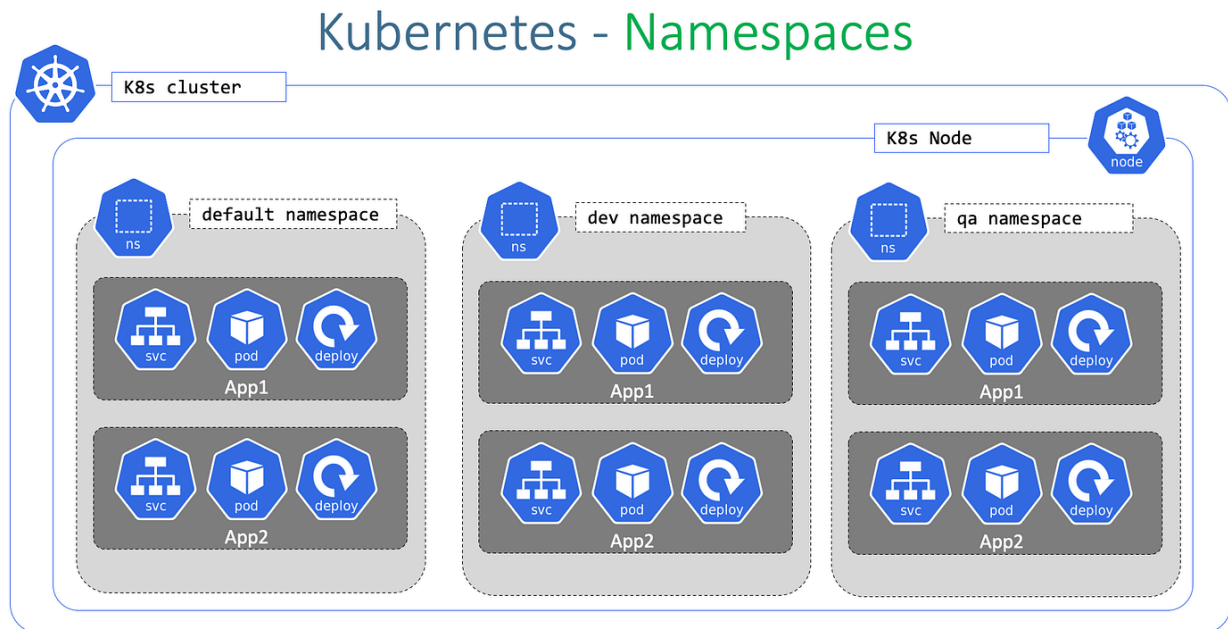
Mars 2026



Phase 1 : Les Fondations	2
Phase 2 : Réseau local et Métriques	3
Phase 4 : Les Secrets	4
Phase 5 : Le coeur de métier (app E-commerce)	17
Phase 6 : Le Verrouillage (Zero Trust)	28
Phase 7 : L'Observabilité (Monitoring)	34
Phase 8 : Automatisation & Résilience	43
Phase 9 : L'Audit Continu (SecOps & Conformité)	64

Phase 1 : Les Fondations

Un cluster kubernetes est composé de un ou plusieurs Node (machines), dans lequel se trouvent différentes ressources parmi lesquels le pod (hébergeant un ou plusieurs conteneurs).



La notion de Namespace dans kubernetes permet d'isoler certains environnements, comme le frontend et la database dans notre cas. Ceci permet également une meilleure gestion globale.

On créera les namespaces suivants pour notre projet:

- backend
- cert-manager
- external-secrets
- Frontend
- database
- observ-prod
- security

D'autres seront plus tard créés par les outils que nous installerons.

Afin de bouger de namespace en namespace pour vérifier les configurations, nous avons utilisés kubens <https://github.com/ahmetb/kubectx>

On peut vérifier via kubens dans quel namespace nous nous trouvons.

Nous allons ensuite activer la collecte des métriques de notre serveur:

<https://github.com/kubernetes-sigs/metrics-server>

Cela nous sera utile plus tard.

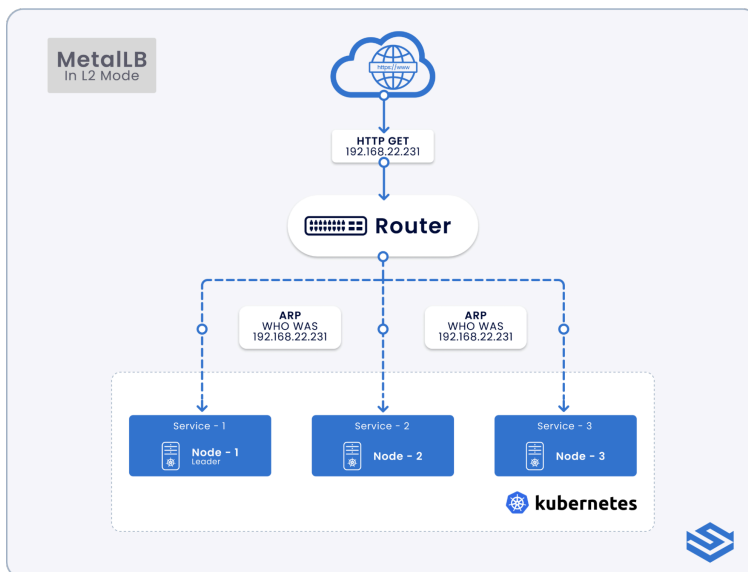
On constate que notre machine consomme actuellement 1519Mi de mémoire.

```
etudiant@debian:~/projet_kubernetes$ kubectl top nodes
NAME          CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
debian        132m         3%       1519Mi          12%
```

100M cpu = 0.1cpu utilisé

Phase 2 : Réseau local et Métriques

Nous allons utiliser MetalLB afin de simuler un load balancer en entrée, comme si c'était une vraie architecture dans le cloud. Lorsqu'un utilisateur externe voudra joindre notre site web, l'ip alloué par metalLB sera contacté.



Installation:

```
etudiant@debian:~$ kubectl apply -f
```

<https://raw.githubusercontent.com/metallb/metallb/v0.15.3/config/manifests/metallb-native.yaml>

```
etudiant@debian:~/projet_kubernetes$ kubectl apply -f metallb-system
✔Active namespace is "metallb-system"
etudiant@debian:~/projet_kubernetes$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
controller-57fdb754bc-qzdgw         1/1     Running   0           26s
speaker-54685                        1/1     Running   0           26s
```

Tout d'abord, nous devons sélectionner des adresses IP utilisées sur le réseau afin que MetalLB puisse les utiliser.

On spécifie l'ip qu'il utilisera:

```
etudiant@debian:~/projet_kubernetes/metallb$ cat adresspool.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: addresspool
  namespace: metallb-system
spec:
  addresses:
  - 192.168.1.202/32
  - 192.168.1.203/32
  - 192.168.1.204/32
  - 192.168.1.205/32
  - 192.168.1.206/32
```

Utile pour l'ARP (L2):

```
! l2advert.yaml
1  apiVersion: metallb.io/v1beta1
2  kind: L2Advertisement
3  metadata:
4    name: advert
5    namespace: metallb-system
6  spec:
7    ipAddressPools:
8      - addresspool
```

On apply

```
etudiant@debian:~/projet_kubernetes/metallb$ kubectl apply -f .
```

Ceci voudra dire que dès qu'un service avec le type LoadBalancer est lancé sur notre Cluster, il récupèrera automatiquement une IP du pool

Phase 4 : Les Secrets

On se place sur le namespace security

```
etudiant@debian:~/projet_kubernetes/nginx$ kubens security
✓Active namespace is "security"
```

On installe ESO (ExternalSecretOperator):

```
etudiant@debian:~/projet_kubernetes$ helm repo add external-secrets https://charts.external-secrets.io
"external-secrets" has been added to your repositories
etudiant@debian:~/projet_kubernetes$ helm install external-secrets \
  external-secrets/external-secrets \
  -n security
NAME: external-secrets
LAST DEPLOYED: Sat Feb 28 07:24:06 2026
NAMESPACE: security
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
```

ESO permettra de faire un pont entre notre secret manager et notre cluster K8S.

On installe hashicorp vault :

```
etudiant@debian2:~$ wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share
/keyrings/hashicorp-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gp
g] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_relea
se -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install vault
```

On vérifie qu'il a bien été installé:

```
etudiant@debian2:~$ vault
Usage: vault <command> [args]
```

Nous allons maintenant sécuriser la communication par chiffrement TLS, afin que l'interface graphique de Vault soit chiffrée. Pour ce faire nous allons devenir notre propre autorité de certification.

On crée le dossier suivant sur une autre machine. On met les permissions 700 pour que seul l'utilisateur etudiant (notre admin) puisse travailler dessus.

```
etudiant@debian2:~$ mkdir ~/pki
etudiant@debian2:~$ chmod 700 ~/pki
```

On génère Clé privé puis publique:

```
etudiant@debian2:~/pki$ openssl genrsa -out ca.key 4096
etudiant@debian2:~/pki$ openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt -subj
"/CN=Root-ca"
```

```

/CN=vault-ca
etudiant@debian2:~/pki$ openssl genrsa -out vault.key 4096
etudiant@debian2:~/pki$ openssl req -new -key vault.key -out vault.csr -subj "/CN=vault.security.local
"

```

On crée san.cnf avec l'ip de la machine:

```

[alt_names]
IP.1 = 192.168.1.168
DNS.1 = vault.security.local
DNS.2 = localhost
IP.2 = 127.0.0.1

```

```

etudiant@debian2:~/pki$ ls -l
total 20
-rw-rw-r-- 1 etudiant etudiant 1801 Feb 28 08:06 ca.crt
-rw----- 1 etudiant etudiant 3446 Feb 28 08:04 ca.key
-rw-rw-r-- 1 etudiant etudiant 267 Feb 28 08:14 san.cnf
-rw-rw-r-- 1 etudiant etudiant 1598 Feb 28 08:10 vault.csr
-rw----- 1 etudiant etudiant 3446 Feb 28 08:09 vault.key

```

On utilise San car les navigateurs modernes ne se basent plus sur le CN mais le SAN

```

etudiant@debian2:~/pki$ openssl x509 -req -in vault.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
vault.crt -days 365 -sha256 -extfile san.cnf
Certificate request self-signature ok
subject=CN=vault.security.local
Enter pass phrase for ca.key:

```

On crée le dossier correspondant pour le tls et on envoie les fichiers:

```

etudiant@debian2:~/pki$ sudo mkdir -p /etc/vault.d/tls/

```

```

etudiant@debian2:~/pki$ cp vault.crt vault.key ca.crt /etc/vault.d/tls/

```

```

etudiant@debian2:~/pki$ sudo cp vault.crt vault.key ca.crt /etc/vault.d/tls/

```

```

etudiant@debian2:~/pki$ ls /etc/vault.d/tls/
ca.crt  vault.crt  vault.key

```

On modifie /etc/vault.d/vault.hcl pour qu'il écoute toutes les interfaces et on spécifie les fichiers précédents:

```
listener "tcp" {
  address           = "0.0.0.0:8200"
  tls_disable       = 0
  tls_cert_file     = "/etc/vault.d/tls/vault.crt"
  tls_key_file      = "/etc/vault.d/tls/vault.key"
}
```

```
etudiant@debian2:~/pki$ sudo systemctl restart vault
```

On peut se rendre sur le site :

Non sécurisé <https://192.168.1.168:8200/ui/vault/init>

Lecteur du certificat : vault.security.local

Général Détails

Émis pour

Nom commun (CN)	vault.security.local
Organisation (O)	<Ne fait pas partie du certificat>
Unité d'organisation (OU)	<Ne fait pas partie du certificat>

Émis par

Nom commun (CN)	Root-ca
Organisation (O)	<Ne fait pas partie du certificat>
Unité d'organisation (OU)	<Ne fait pas partie du certificat>

Durée de validité

Émis le	samedi 28 février 2026 à 08:39:15
Expire le	dimanche 28 février 2027 à 08:39:15

Empreintes SHA-256

Certificat	68d8328b8a1e74f72163c37e954a149135f53d2685cef67a7dd3ce1080dfdc40
Clé publique	2276529858496305daaed2c8ed38742c1a3e07af077fabe014bdd5b01241241e

Au démarrage de Vault, 5 clés seront partagées, et il faut en réunir au moins 3 pour déverrouiller le coffre fort. On peut configurer cela pour en demander plus ou moins.

Let's set up the initial set of root keys that you will need in case of an emergency.

Key shares

The number of key shares to split the root key into

Key threshold

The number of key shares required to reconstruct the root key

- ✓ [Encrypt output with PGP](#)
- ✓ [Encrypt root token with PGP](#)

Initialize

Vault has been initialized! Here are your 5 keys.

Please securely distribute the keys below. When the Vault is re-sealed, restarted, or stopped, you must provide at least **3** of these keys to unseal it again. Vault does not store the root key. Without at least **3** keys, your Vault will remain permanently sealed.

Initial root token

Key 1

Key 2

Key 3

Key 4

Key 5

[Continue to Unseal](#)

[Download keys](#)

On stocke les clés dans un gestionnaire de password sécurisés

Sur l'écran on fournit au moins 3 clés:

Unseal Vault

Vault is sealed

Unseal Vault by entering portions of the unseal key. This can be done via multiple mechanisms on multiple computers. Once all portions are entered, the root key will be decrypted and Vault will unseal.

Unseal Key Portion

.....

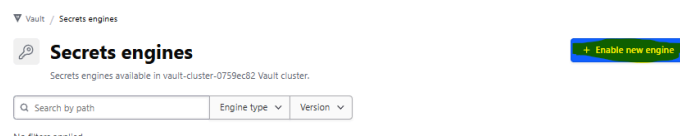
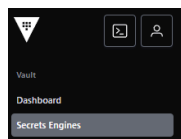
Unseal

2/3 keys provided 

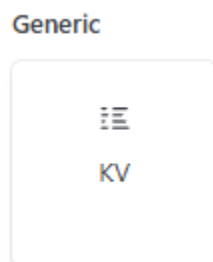
On va utiliser le root token seulement pour la configuration initiale puis on ne l'utilisera plus:

Ajoutons nos secrets.

On sélectionne "New engine":



On choisit kv (key:value) , ensuite on laisse les options par défaut:



Maximum number of versions

0

If checked, all keys will require the `cas` parameter to be set on all write requests. A key's metadata settings can overwrite this value.

A secret's version must be manually deleted.

Enable engine

[Back](#)

[Secrets](#) / [secret](#) / [Create](#)

☐ JSON

Names with forward slashes define hierarchical path structures.

app-prod/database

backend_user

[illegible]

backend_password

[illegible]

Add

▼ Show secret metadata

Save

Cancel

11

```
etudiant@debian2:~/vault$ cat eso-policy.hcl
path "secret/data/app-prod/database" {
  capabilities = ["read"]
}
```

On exécute mais on rencontre l'erreur:

```
etudiant@debian2:~/vault$ vault policy write eso-policy eso-policy.hcl
WARNING! VAULT_ADDR and -address unset. Defaulting to https://127.0.0.1:8200.
Error uploading policy: Put "https://127.0.0.1:8200/v1/sys/policies/acl/eso-policy": tls: failed to verify certificate: x509: certificate signed by unknown authority
```

On lance nano ~/.bashrc et on ajoute comme variables d'environnement la clé publique et l'adresse du serveur:

```
export VAULT_ADDR="https://192.168.1.168:8200"
export VAULT_CACERT="/etc/vault.d/tls/ca.crt"
```

Puis on tape source ~/.bashrc afin d'éviter de les taper à chaque redémarrage du serveur

<https://developer.hashicorp.com/vault/docs/auth/approle>

On valide la policy, mais il faut nous login à vault avant cela.

Nous allons créer un approle pour l'ESO, ce sera un rôle qu'utilisera ESO pour demander les secrets stockés dans le hashicorp Vault.

```
etudiant@debian2:~/vault$ vault auth enable approle
Success! Enabled approle auth method at: approle/
```

On crée ensuite l'approle qu'utilisera ESO avec les spécifications suivantes:

On peut alors afficher le Secretid:

```
etudiant@debian2:~/vault$ vault write auth/approle/role/eso-role token_policies="eso-policy" secret_id_ttl=0 token_ttl=1h token_max_ttl=3h
Success! Data written to: auth/approle/role/eso-role
etudiant@debian2:~/vault$ vault write -f auth/approle/role/eso-role/secret-id
Key      Value
---      -
secret_id 59c7debc-59e0-f756-7faf-958aea32cbf9
```

On peut ensuite afficher le Roleid:

```
etudiant@debian2:~/vault$ vault read auth/approle/role/eso-role/role-id
Key      Value
---      -
role_id  ff16f9b1-ec97-488a-0be0-62080e761b79
```

Le rôle est lié à un token qui a une durée d'1h.

On pourrait rajouter un ttl pour le secret_id mais cela bloquerait ESO, et empêcherait notre cluster de bien fonctionner. Idéalement il faut un système qui effectue une rotation de secretID.

On les note dans un gestionnaire de mot de passe sécurisé.

Retournons à notre cluster Kubernetes et créons un secret

<https://kubernetes.io/fr/docs/concepts/configuration/secret/>

On encode en base64 roleId et SecretId:

```
etudiant@debian2:~/vault$ echo "ff16f9b1-ec97-488a-0be0-62080e761b79" | base64
ZmYxNmY5YjEtZW5Ny000DhhLTBiZTAaNjIwODBlNzYxYjc5Cg==
etudiant@debian2:~/vault$ echo "59c7debc-59e0-f756-7faf-958aea32cbf9" | base64
NTljN2RlYmMtNTlLMC1mNzU2LTdmYWYtOTU4YWVhMzJjYmY5Cg==
```

```
etudiant@debian:~/projet_kubernetes/vault$ cat vault_id.yaml
apiVersion: v1
kind: Secret
metadata:
  name: vault-approlesecret
  namespace: security
data:
  roleId: ZmYxNmY5YjEtZW5Ny000DhhLTBiZTAaNjIwODBlNzYxYjc5Cg==
  secretId: NTljN2RlYmMtNTlLMC1mNzU2LTdmYWYtOTU4YWVhMzJjYmY5Cg==
```

On importe le certificat sur notre machine car kubernetes ne connaît pas la CA.

```
etudiant@debian:~/projet_kubernetes/other$ ls
ca.crt
```

On crée le secret contenant le certificat d'autorité

```
etudiant@debian:~/projet_kubernetes/other$ kubectl create secret generic vault-ca-cert --from-file=ca.
crt=/home/etudiant/projet_kubernetes/other/ca.crt -n security
secret/vault-ca-cert created
```

Désormais nous allons passer au ESO, qui est le pont entre notre cluster le vault.

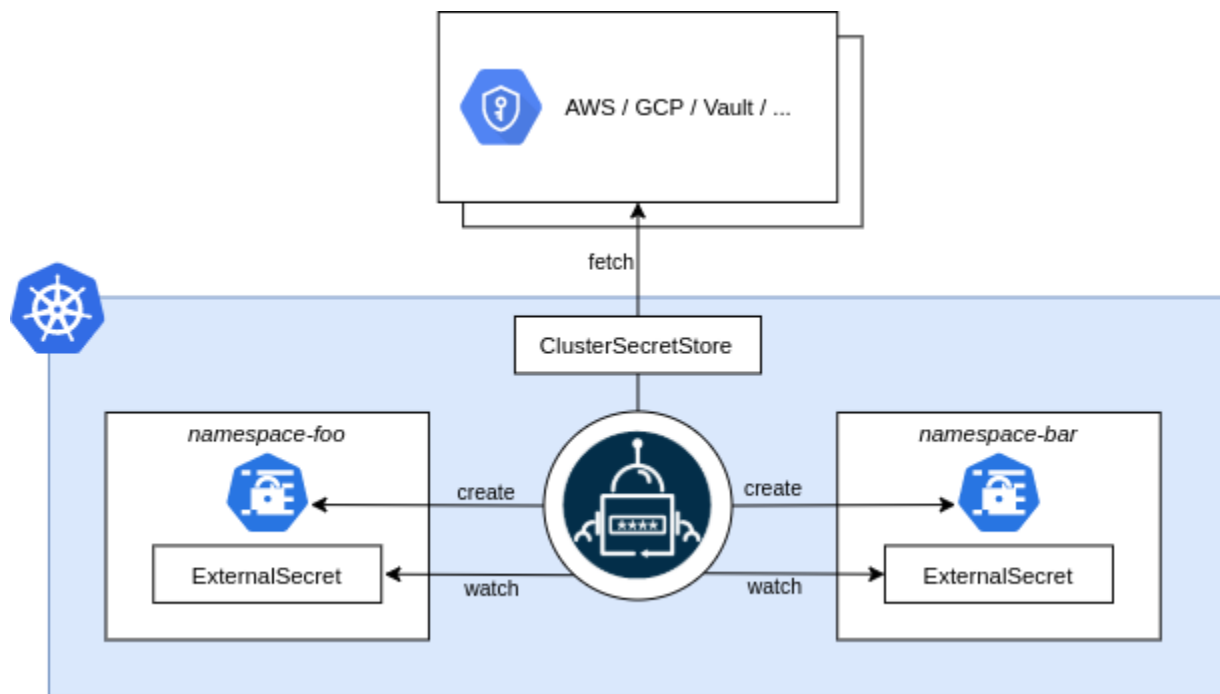
<https://external-secrets.io/latest/provider/hashicorp-vault/>

On télécharge ESO:

helm repo add external-secrets <https://charts.external-secrets.io>

```
helm install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets \
  --create-namespace \
  --set installCRDs=true
```

```
helm upgrade external-secrets \ external-secrets/external-secrets \ -n external-secrets \ --set
installCRDs=true
```



Pour fonctionner, ESO utilise dans notre cas:

- ClusterSecretStore, qui récupère les secrets directement depuis Vault.
- ExternalSecret qui spécifie quoi chercher et comment chercher

<https://external-secrets.io/latest/api/secretstore/>

Il y a également possibilité de faire du mTLS où ESO aussi a un certificat (self CA)

On doit faire un clustersecretstore car nos secrets dans vault seront utilisés par des namespace différents, et c'est une meilleur pratique de sécurité de ne pas mélanger secrets et code.

<https://external-secrets.io/latest/api/clustersecretstore/>

On définit notamment l'adresse du server, son certificat, et le rôle associé créée tout à l'heure permettant à ESO de lire seulement les secrets.

```
etudiant@debian:~/projet_kubernetes/vault$ cat cluster_secret_store.yaml
apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: vault-clustersecretstore

spec:
  provider:
    vault:
      server: "https://192.168.1.168:8200"
      path: "secret"
      version: "v2"

      caProvider:
        type: "Secret"
        name: "vault-ca-cert"
        key: "ca.crt"
        namespace: "security"

      auth:
        appRole:
          path: "approle"
          roleRef:
            name: "vault-approlesecret"
            namespace: "security"
            key: "roleId"
          secretRef:
            name: "vault-approlesecret"
            namespace: "security"
            key: "secretId"
```

On apply.

Désormais on doit appliquer le vault-externalsecret , qui est spécifique à Vault.

<https://external-secrets.io/latest/provider/hashicorp-vault/>

On lui indique où chercher dans vault, et quoi chercher.

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: vault-externalsecret
  namespace: backend
spec:
  refreshInterval: "15s"
  secretStoreRef:
    name: vault-clustersecretstore
    kind: ClusterSecretStore
  target:
    name: backend-secrets
    creationPolicy: Owner
  data:
  - secretKey: db_user
    remoteRef:
      key: app-prod/database
      property: db_user
  - secretKey: db_password
    remoteRef:
      key: app-prod/database
      property: db_password
```

On apply et on constate:

```
etudiant@debian:~/projet_kubernetes/backend$ kubectl apply -f externalsecret.yaml
externalsecret.external-secrets.io/vault-externalsecret configured
etudiant@debian:~/projet_kubernetes/backend$ kubectl get secret
NAME                TYPE      DATA   AGE
backend-secrets     Opaque    2       2s
```

Nous avons bien la data:

```
etudiant@debian:~/projet_kubernetes/vault$ kubectl get secret backend-secrets -n backend -o jsonpath='{.data.db_user}' | base64 -d; echo
db_user

etudiant@debian:~/projet_kubernetes/backend$ kubectl get externalsecret
NAME                STORETYPE      STORE                REFRESH INTERVAL   STATUS
vault-externalsecret ClusterSecretStore vault-clustersecretstore 15s                 SecretSynced
True
```

Seule bémole, cette information est codée en base64. Un hacker pourrait la consulter et saboter notre infrastructure kubernetes. Il faudrait alors appliquer le principe d'encryption at rest

(encryption au repos) de la base etcd. Pour information, kubernetes stocke les secrets dans sa base etcd.

Caution:

Kubernetes Secrets are, by default, stored unencrypted in the API server's underlying data store (etcd). Anyone with API access can retrieve or modify a Secret, and so can anyone with access to etcd. Additionally, anyone who is authorized to create a Pod in a namespace can use that access to read any Secret in that namespace; this includes indirect access such as the ability to create a Deployment.

Cf: <https://kubernetes.io/docs/concepts/configuration/secret/>

Il existe un article dédié à cela, mais c'est très complexe:

<https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>

Phase 5 : Le coeur de métier (app E-commerce)

Passons à l'application e-commerce.

Créons les dossiers requis:

```
etudiant@debian:~/projet_kubernetes$ mkdir frontend && cd frontend
```

Dockerfile du frontend:

```

etudiant@debian:~/projet_kubernetes/frontend$ cat Dockerfile

FROM nginxinc/nginx-unprivileged:alpine-slim

COPY ./app/default.conf /etc/nginx/conf.d/default.conf

COPY ./app//html /usr/share/nginx/html

USER 101

EXPOSE 8080

CMD ["nginx", "-g", "daemon off;"]

```

On utilise une image sans privilèges, très sécurisé. L'USER 101 est l'utilisateur nginx, ça évite de lancer ça en tant que root.

```

etudiant@debian:~/projet_kubernetes/frontend$ ls
app deployment.yaml Dockerfile frontend-svc.yaml

```

Le backend:

```

etudiant@debian:~/projet_kubernetes/backend$ cat Dockerfile

FROM node:24-alpine AS builder

WORKDIR /usr/src/app
COPY ./app/package*.json ./
RUN npm ci --omit=dev
COPY ./app .

FROM gcr.io/distroless/nodejs24-debian13

ENV NODE_ENV=production

WORKDIR /usr/src/app

COPY --from=builder --chown=nonroot:nonroot /usr/src/app /usr/src/app

EXPOSE 3000

CMD ["server.js"]

```

On utilise le registre GCR de google pour récupérer une image nodejs distroless, c'est-à-dire qu'elle ne contient que l'application et ses dépendances.

```
etudiant@debian:~/projet_kubernetes/backend$ ls
app  backend-svc.yaml  deployment.yaml  Dockerfile  externalsecret.yaml
```

L'Image backend utilisé le docker multistage-build afin d'être encore plus sécurisé.

On va les stocker en local dans notre propre registre. Pour cela on utilise registry
https://hub.docker.com/_/registry

```
etudiant@debian:~/projet_kubernetes/frontend$ docker run -d -p 5000:5000 --restart always --name registry registry:3
```

Il écoute bien au port 5000:

```
etudiant@debian:~/projet_kubernetes/frontend$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
0ce6f3fb487d   registry:3    "/entrypoint.sh /etc..." About a minute ago Up About a minute 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
NAME          registry
```

On push

```
etudiant@debian:~/projet_kubernetes/frontend$ docker tag frontend:v1 localhost:5000/frontend:v1
etudiant@debian:~/projet_kubernetes/frontend$ docker push localhost:5000/frontend:v1

etudiant@debian:~/projet_kubernetes/backend$ docker tag backend:v1 localhost:5000/backend:v1
etudiant@debian:~/projet_kubernetes/backend$ docker push localhost:5000/backend:v1
```

On doit autoriser K3s à lire ce registry:

<https://docs.k3s.io/installation/private-registry>

On doit alors modifier /etc/rancher/k3s/registries.yaml

```
etudiant@debian:~/projet_kubernetes/frontend$ cat /etc/rancher/k3s/registries.yaml
mirrors:
  "192.168.1.61:5000":
    endpoint:
      - "http://192.168.1.61:5000"

configs:
  "192.168.1.61:5000":
    tls:
      insecure_skip_verify: true
```

```
etudiant@debian:~/projet_kubernetes/backend$ curl http://localhost:5000/v2/_catalog
{"repositories":["backend", "frontend"]}
```

Deployment backend:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: 192.168.1.61:5000/backend:v1.6
          envFrom:
            - configMapRef:
                name: backend-configmap
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: backend-secrets
                  key: db_user
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: backend-secrets
                  key: db_password
```

On y spécifie alors une configmap pour expliquer comment joindre la db:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-configmap
  namespace: backend
data:
  DB_HOST: postgres-svc.database.svc.cluster.local
  DB_NAME: shopdb
  DB_PORT: "5432"
```

Svc du backend qui écoute au port 3000:

```
etudiant@debian:~/projet_kubernetes/backend$ cat backend-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: backend
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
```

On apply:

```
etudiant@debian:~/projet_kubernetes/backend$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-7b5968676f-6nm2s           1/1     Running   0           35s
etudiant@debian:~/projet_kubernetes/backend$ kubectl logs backend-7b5968676f-6nm2s
Backend E-commerce en écoute sur le port 3000
Utilisateur BDD injecté par Vault : backend_user
```

Frontend:

On spécifie le port 80 en réception, mais le port réel sur lequel écoute le conteneur est le port 8080.

```

apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
  namespace: frontend
spec:
  type: ClusterIP
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080

```

On utilise un ingress pour indiquer à traefik que pour tout traffic visant l'host shop.local, il doit être redirigé vers le service du frontend.

```

etudiant@debian:~/projet_kubernetes/frontend$ cat ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
  namespace: frontend
spec:
  ingressClassName: traefik
  rules:
    - host: shop.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-svc
                port:
                  number: 80

```

```

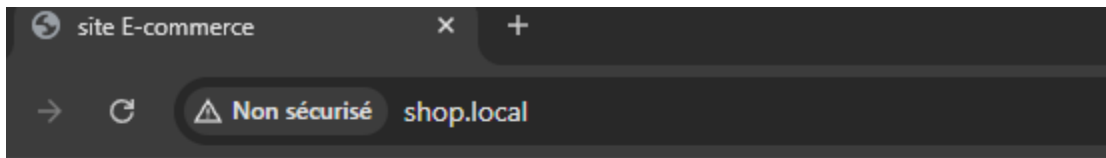
etudiant@debian:~/projet_kubernetes/frontend$ kubectl get ingress -n frontend
NAME                CLASS    HOSTS          ADDRESS          PORTS    AGE
frontend-ingress    traefik  shop.local     192.168.1.202    80       20m
etudiant@debian:~/projet_kubernetes/frontend$ kubectl get svc -n kube-system -l app.kubernetes.io/name=traefik
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)                                     AGE
traefik   LoadBalancer  10.43.158.87   192.168.1.202  80:32624/TCP,443:30243/TCP                3d19h

```

On ajoute l'entrée dns

```
192.168.1.202 shop.local
```

On apply et on peut y accéder:



welcome

Vault and Kubernetes.

[Home](#) | [About](#) | [Products](#)

Backend is coming

Postgres ExternalSecrets:

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: database-secrets
  namespace: database
spec:
  refreshInterval: "15s"
  secretStoreRef:
    name: vault-clustersecretstore
    kind: ClusterSecretStore
  target:
    name: database-secrets
  data:
  - secretKey: POSTGRES_USER
    remoteRef:
      key: app-prod/database
      property: db_user
  - secretKey: POSTGRES_PASSWORD
    remoteRef:
      key: app-prod/database
      property: db_password
```



```
etudiant@debian:~/projet_kubernetes/backend$ kubectl get secret database-secrets -n database -o jsonpath='{.data.POSTGRES_USER}' | base64 -d; echo  
db_user
```

On va injecter ce script qui créera la table product avec des valeurs:

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: postgres-init-script  
  namespace: database  
data:  
  01-init.sql: |  
    CREATE TABLE IF NOT EXISTS products (  
      id SERIAL PRIMARY KEY,  
      name VARCHAR(100) NOT NULL,  
      description TEXT,  
      price DECIMAL(10, 2) NOT NULL,  
      stock INTEGER DEFAULT 0  
    );  
  
    INSERT INTO products (name, description, price, stock) VALUES  
    ('T-Shirt Kubernetes', 'Le vêtement officiel des administrateurs.', 25.99, 100),  
    ('Mug HashiCorp Vault', 'Pour garder vos secrets au chaud.', 15.50, 50),  
    ('Sticker DevSecOps', 'Fait fuir les hackers.', 2.99, 500);
```

Toutes les images PostgreSQL officielles sont programmées pour qu'au tout premier démarrage (quand le dossier /var/lib/postgresql/data est vide), Postgres cherche des fichiers .sql ou .sh dans le dossier /docker-entrypoint-initdb.d/.

Statefulset database:

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: database
spec:
  serviceName: "postgres-svc"
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      securityContext:
        fsGroup: 70
      containers:
        - name: postgres
          image: postgres:16-alpine
          securityContext:
            runAsUser: 70
            runAsGroup: 70
            runAsNonRoot: true
          env:
            - name: POSTGRES_DB
              value: "shopdb"
            - name: PGDATA
              value: "/var/lib/postgresql/data/pgdata"
          envFrom:
            - secretRef:
                name: database-secrets
          ports:
            - containerPort: 5432

```

<https://artifacthub.io/packages/helm/cloudpirates-postgres/postgres>

Il est recommandé d'utiliser fsGroup 70 pour la sécurité.

Voici comment cela fonctionne pour PostgreSQL :

Dans l'image Docker officielle de PostgreSQL, l'utilisateur s'appelle postgres et son identifiant (UID) est 70. Son groupe principal (GID) est aussi 70.

Quand Kubernetes monte un volume , ce disque appartient souvent par défaut à l'utilisateur root (ID 0). Si PostgreSQL (ID 70) essaie d'écrire dedans, il reçoit une erreur "Permission denied".

En ajoutant fsGroup: 70 on demande à Kubernetes qu'au moment où on monte le volume, il change le groupe propriétaire de tous les fichiers pour que ce soit le groupe 70".

On crée les volumes associés pour le stockage de données, le script d'initialisation. On demande 1giga en lecture/écriture.

```
volumeMounts:
- name: postgres-storage
  mountPath: /var/lib/postgresql/data
- name: init-script
  mountPath: /docker-entrypoint-initdb.d/01-init.sql
  subPath: 01-init.sql
volumes:
- name: init-script
  configMap:
    name: postgres-init-script
volumeClaimTemplates:
- metadata:
    name: postgres-storage
  spec:
    accessModes: ["ReadWriteOnce"]
    resources:
      requests:
        storage: 1Gi
```

```
etudiant@debian:~/projet_kubernetes/database$ kubectl get pvc
NAME                                STATUS  VOLUME                                     CAPACITY  ACCESS MO
DES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
postgres-storage-postgres-0        Bound   pvc-cbbec87f-6e30-4f1f-a28d-e618ebf5b4c4  1Gi       RWO
local-path                         <unset> 88s
```

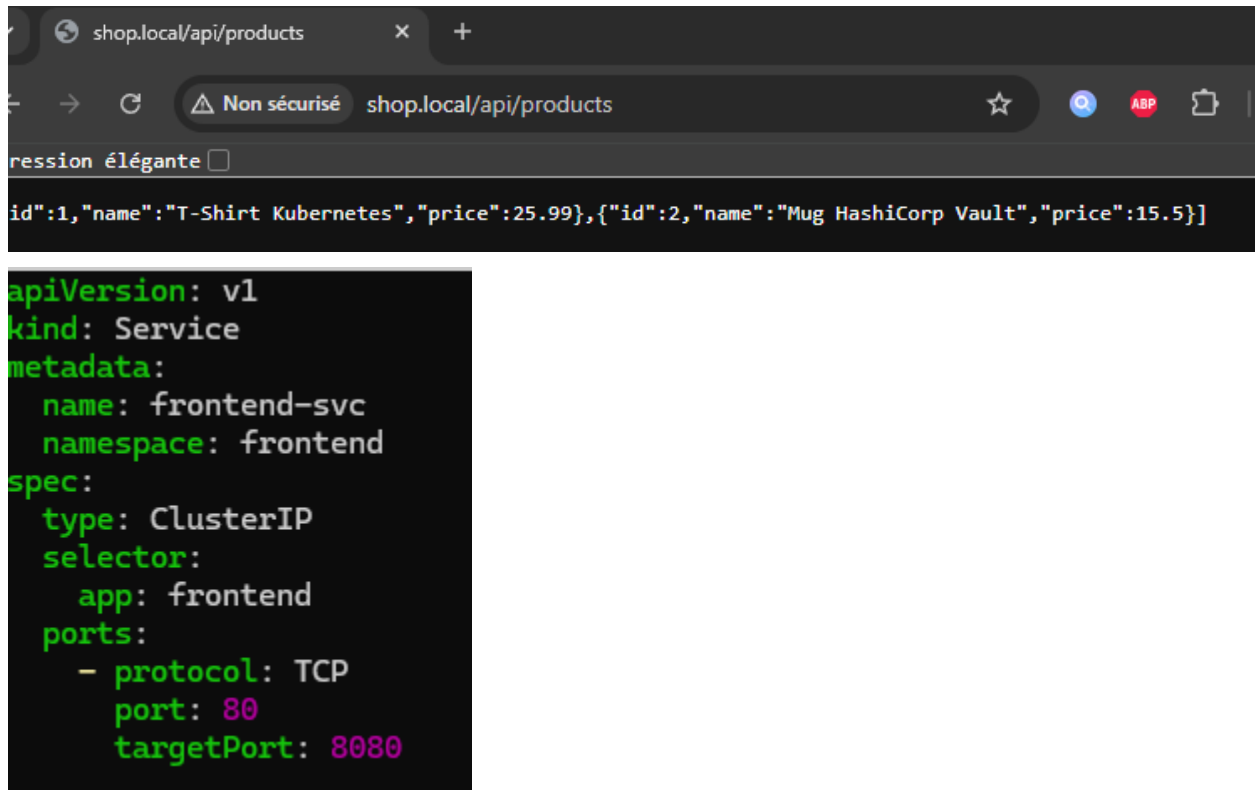
C'est parfait:

```
etudiant@debian:~/projet_kubernetes/database$ kubectl exec -it postgres-0 -n database -- psql -U db_us
er -d shopdb
psql (16.13)
Type "help" for help.

shopdb=# select *;
ERROR:  SELECT * with no tables specified is not valid
LINE 1: select *;
        ^

shopdb=# select * from products;
 id |      name      |      description      | price | stock
----+-----+-----+-----+-----
  1 | T-Shirt Kubernetes | Le vêtement officiel des administrateurs. | 25.99 |   100
  2 | Mug HashiCorp Vault | Pour garder vos secrets au chaud. | 15.50 |    50
  3 | Sticker DevSecOps | Fait fuir les hackers. | 2.99 |   500
(3 rows)

shopdb=#
```



Dans Kubernetes, l'Ingress Controller est un programme intelligent qui lit en permanence l'API de Kubernetes pour voir si de nouvelles règles de routage (les objets Ingress) ont été créées.

Voici ce qui s'est passé dans le cluster :

1. K3s a installé Traefik par défaut. Pour que Traefik puisse recevoir du trafic depuis l'extérieur, K3s a créé pour lui un Service de type LoadBalancer.
2. L'attente : Sans MetalLB, ce service serait resté bloqué indéfiniment avec le statut `<pending>` dans la colonne EXTERNAL-IP, car Kubernetes ne sait pas fabriquer des adresses IP publiques/locales tout seul.
3. L'intervention de MetalLB : Dès qu'on a configuré MetalLB à la Phase 2 (avec la plage d'IP 192.168.1.x), le contrôleur de MetalLB s'est mis à écouter le cluster.
4. MetalLB a vu : *"Traefik demande une IP de type LoadBalancer"*, Il a donc pioché une adresse IP disponible dans la plage donnée et l'a "offerte" (assignée) au service Traefik.
5. L'annonce (L2Advertisement) : Enfin, MetalLB a crié sur le réseau physique (via le protocole ARP) : *"L'adresse IP de Traefik est ici, envoyez-moi le trafic !"*.

Phase 6 : Le Verrouillage (Zero Trust)

<https://kubernetes.io/docs/concepts/services-networking/network-policies/>

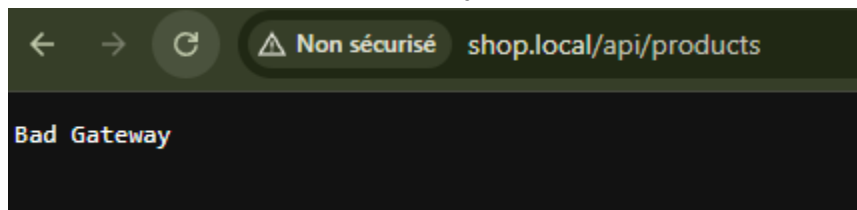
Afin d'éviter que chaque pod puisse parler avec tout le monde, il faut segmenter le réseau via des politiques réseaux (network policies).

On crée la policy par défaut pour les namespace qui consiste à tout bloquer (default-deny-all):

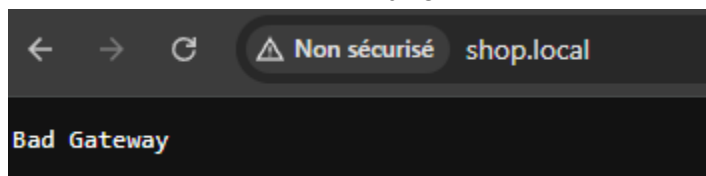
```
etudiant@debian:~/projet_kubernetes/frontend$ cat default-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: frontend/backend/database
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

On bloque tout par défaut.

Désormais le frontend ne peut plus joindre le backend



Evidemment le site n'est plus joignable car le frontend n'autorise pas traefik



On autorise alors le trafic provenant du traefik situé dans le namespace kube-system dans notre distribution k3s, et celui à destination du DNS interne de K8S pour résoudre les noms des services. Le port 53 DNS utilise TCP et UDP.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-traefik
  namespace: frontend
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - protocol: TCP
          port: 8080
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: backend
      ports:
        - protocol: TCP
          port: 3000

```

Désormais shop.local est résolu:

welcome

Vault and Kubernetes.

[Home](#) | [About](#) | [Products](#)

Backend is coming

<https://oneuptime.com/blog/post/2026-02-02-kubernetes-egress-network-policies/view>

On ne peut pas encore contacter le backend car il ne nous a pas encore autorisés en trafic d'entrée (ingress).

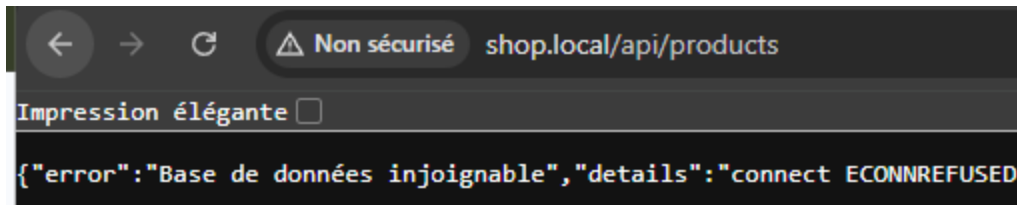
Pour le backend, on autorise le frontend en entrée (ingress) et il doit pouvoir sortir vers la bdd (egress):

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: frontend
    ports:
    - protocol: TCP
      port: 3000
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: database
    ports:
    - protocol: TCP
      port: 5432

```

Désormais le frontend peut joindre le backend, mais le backend ne peut pas joindre la bdd:

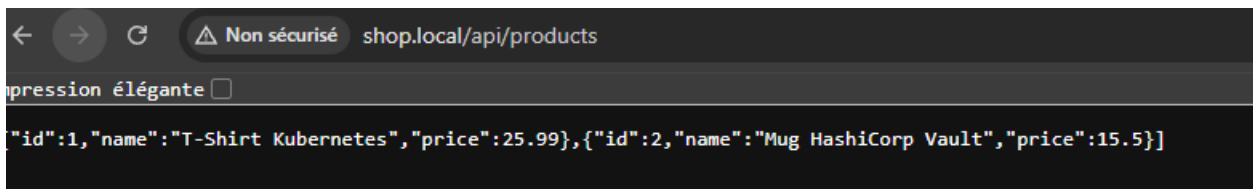


Pour la db, on autorise seulement le backend en entrée. Nul besoin d'ajouter de règles en egress car la BDD n'est pas à l'initiative de communications extérieures.:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend
  namespace: database
spec:
  podSelector:
    matchLabels:
      app: postgres
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: backend
    ports:
    - protocol: TCP
      port: 5432
```

Les networkpolicies sont stateful; pour chaque connexion entrée, le retour est autorisé.

Désormais ça fonctionne:



Par défaut, Kubernetes utilise un service account par défaut appelé "default". Bémole, son token est écrit en clair dans /var/run/secrets :

```
student@debian:~/projekt_kubernetes/database$ kubectl exec -it pods/frontend-899c-f9b5b-6z2zg -- /bin/sh
/ $ car /var/run/secrets/
/bin/sh: car: not found
/ $ ls /var/run/secrets/
kubernetes.io
/ $ cat /var/run/secrets/kubernetes.io/serviceaccount/
..2026_03_07_11_20_33.3817985138/ namespace
..data/ token
ca.crt
/ $ cat /var/run/secrets/kubernetes.io/serviceaccount/token
eyJhbGciOiJIUzI1NiIsImtpZCI6ImxnSGxTd2hzeGdoOVBRcklHMLAyxY1JMKVZyWE2NWs4bC1LejVBO
TY1VWcifQ.eyJhdWQiOiJsiaHR0cHM6Ly9rdWJlcmlldGVzLmRlZmF1bHQuc3ZjLmNsdXN0ZXIubG9jYWw
iLCJRmM3MiXSwiZXhwIjojODAwNDU0NDMzMzYXQiojE3NzI4ODIOMzMzImIzcyI6Imh0dHBzOj8va3Vi
ZXJuZXRlcy5kZWZhdx0LnN2Yy5jbHVzdGVyLmxvY2FsIiwianRpIjojY2NlY2NiZjItYWVhZC00OTU2L
WEwNjItZWU0YWUiWzJyYyNGFiIiwia3ViZXJuZXRlcy5pbyI6eyJuYW1lc3BhY2UiOiJmcu9udGVuZCIsIm
5vZGUiOiNsibmFtZSI6ImRlYmhiInVpZCI6IjhjZDkxOTg4LTc1ZTgtNGE5NC04Y2VjLWNkMzk5NGE
zMWl0MCJ9LCJwb2QiOiNsibmFtZSI6ImRlYmZyb250ZW5kLTg5OWNmOWI1Yi02ejJ6ZyIsInVpZCI6ImM5NTcx
MzI3LTJmZjQtNDEwZiliOGMyLWE1ZDdlMzAzMTMzYSJ9LCJzZXJ2aWNlYWNBj3VudCI6eyJuYW1lIjojZ
GVmYXVsdiCI6ImRlYmZyb250ZW5kLTg5OWNmOWI1Yi02ejJ6ZyIsInVpZCI6IjAxOWVjMzY4LTBkYzY4MTNGI1ZC1iY2U3LTQ5ZmZlMTFTfiYzJiNiJ9LCJ3YXJuYW
Z0ZXIiOiE3NzI4ODYwNDNB9LCJuYmYiOiE3NzI4ODIOMzMzInN1YiI6InN5c3RlbTpzZXJ2aWNlYWNBj3V
udDpmcm9udGVuZDpkZWZhdx0In0.f87geENjcu_RcYQDnOLPjybe6GzwqvT4Xvi2tNxjwZwkTX6X-4kv
v4lhH_YzXeJYUTiypdTxbngIjcarX714WefKkr7VLHwHyZgajcQAoudJEbEZBlvNeqX4Js2nuMtZJ5yC
GXLaigv5H6Xgije7Agif_lrtRxkr9KGr0NyDxoWMKmWa0ahiLYjHynwmDJzpoePgwsDnm856g8zKH9yMB
otug471eGvkSDL7c9ek30_RF2VusPjf7Cj3HuZLfTct0erg8whGqafeJwx1l1PeTXGZNYURhBs16dey
jzhDaaB_wOMrEiaQvY7vJUNoMVPwGPOE-NaAKz1PhsYx-aw/ $
```

Ce jeton permet au conteneur de s'authentifier auprès de l'API centrale de Kubernetes. Par chance, par défaut il ne donne pas de privilèges.

Default service accounts

When you create a cluster, Kubernetes automatically creates a ServiceAccount object named `default` for every namespace in your cluster. The `default` service accounts in each namespace get no permissions by default other than the [default API discovery permissions](#) that Kubernetes grants to all authenticated principals if role-based access control (RBAC) is enabled. If you delete the `default` ServiceAccount object in a namespace, the control plane replaces it with a new one.

If you deploy a Pod in a namespace, and you don't manually assign a ServiceAccount to the Pod, Kubernetes assigns the `default` ServiceAccount for that namespace to the Pod.

Bémol, il a accès au “default api discovery permission” Un attaquant qui récupère ton token peut donc :

- Scanner le cluster pour identifier les versions de Kubernetes.
- Identifier les services installés (monitoring, bases de données, etc.).

- Préparer une attaque ciblée en fonction des faiblesses des versions détectées.

On crée un serviceaccount pour le frontend/backend/database et on n'autorise pas à monter le token en clair sur le pod:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: frontend-sa
  namespace: frontend
automountServiceAccountToken: false
```

Et on modifie donc le deployment:

```
spec:
  serviceAccountName: frontend-sa
  containers:
  - name: frontend
```

On apply le nouveau deployment et on constate qu'aucun token n'est visible en clair:

```
etudiant@debian:~/projet_kubernetes/frontend$ kubectl exec -it pods/frontend-58fb
f6f976-mdfhs -- /bin/sh
/ $ ls /var/run/
lock
```

On constate qu'il n'y a plus de dossier secrets.

Phase 7 : L'Observabilité (Monitoring)

Installation prometheus et grafana, qui serviront à observer et alerter.

Installation via helm(le gestionnaire de paquets de Kubernetes) :

helm repo add prometheus-community <https://prometheus-community.github.io/helm-charts>

helm repo update

```
etudiant@debian:~/projet_kubernetes/backend$ kubens observ-prod
✓Active namespace is "observ-prod"
```

On installe la stack prometheus+grafana.

Via Helm , quand on installe une application, on doit lui donner un nom. Ce nom s'appelle une Release (une version installée).

Dans notre cas, il s'agit de monitoring. Nous utiliserons cette information plus tard.

```
etudiant@debian:~/projet_kubernetes/backend$ helm install monitoring prometheus-community/kube-prometheus-stack
NAME: monitoring
LAST DEPLOYED: Sun Mar  8 08:35:27 2026
NAMESPACE: observ-prod
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
```

```
etudiant@debian:~/projet_kubernetes/backend$ kubectl get pods
NAME                                READY   STATUS    RESTARTS
alertmanager-monitoring-kube-prometheus-alertmanager-0  2/2     Running   0
monitoring-grafana-678954d975-55w9g                    3/3     Running   0
monitoring-kube-prometheus-operator-6f56d46845-5qqsp    1/1     Running   0
monitoring-kube-state-metrics-85c77b54f-dqsj8           1/1     Running   0
monitoring-prometheus-node-exporter-wcdsq               1/1     Running   0
prometheus-monitoring-kube-prometheus-prometheus-0      2/2     Running   0
```

On ajoute un ingress pour que traefik redirige le trafic à destination de grafana.local vers le service de grafana

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: grafana-ingress
  namespace: observ-prod
spec:
  ingressClassName: traefik
  rules:
  - host: grafana.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: monitoring-grafana
            port:
              number: 80

```

```

# 58429489418 378deme7e0m
192.168.1.202 shop.local grafana.local

```



Welcome to Grafana

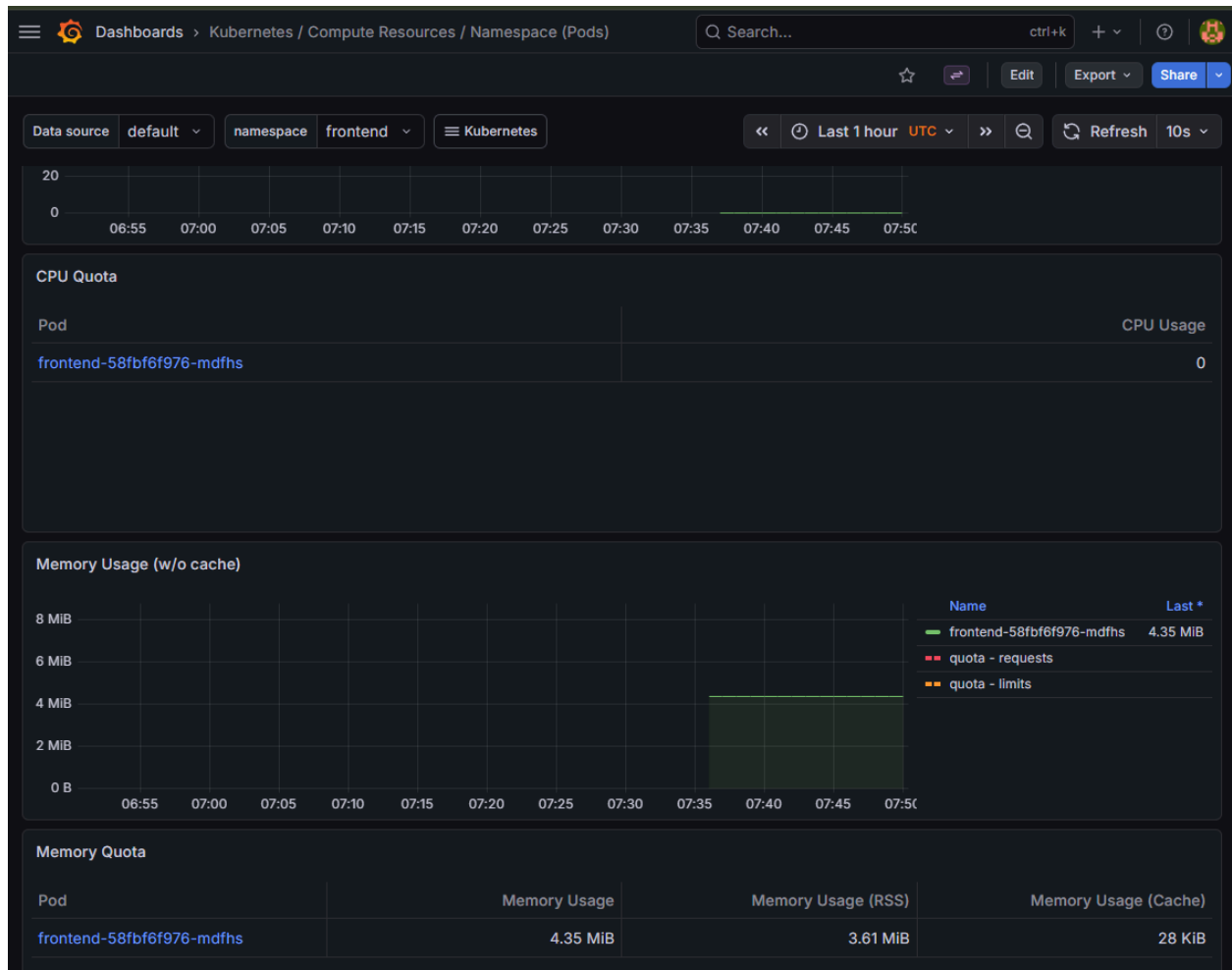
Email or username

Password



Log in

[Forgot your password?](#)



On peut alors ensuite observer les métriques via le dashboard, pour chaque namespace.

Nous allons utiliser l'alert manager afin de nous alerter en cas d'augmentation CPU.

<https://www.alibabacloud.com/help/en/prometheus/use-cases/use-prometheusrule-to-create-prometheus-alert-rules>

Dans Kubernetes, les ressources CPU se mesurent en "cores" (cœurs).

1.0 = 1 cœur CPU complet (soit 1000m ou "millicores").

0.05 = 0,05 cœur CPU, soit 50m (50 millicores).

On ajoute des limites au backend, et on indique quels sont les ressources requises pour qu'il tourne:

```
containers:
- name: backend
  image: 192.168.1.61:5000/backend:v1.9
  resources:
    requests:
      cpu: "100m"
      memory: "100Mi"
    limits:
      cpu: "250m"
      memory: "256Mi"
```

On apply:

```
etudiant@debian:~/projet_kubernetes/backend$ kubectl get pod backend-7786458c8c-l7h5z -n backend -o jsonpath='{.spec.containers[*].resources}'
{"limits":{"cpu":"250m","memory":"256Mi"},"requests":{"cpu":"100m","memory":"100Mi"}}etudiant@debian:~/projet_kubernetes/backend$
```

On crée le role prometheus suivant qui doit alerté lorsque le pod backend dans le namespace backend consomme plus de 80% de sa limite CPU maximale allouée depuis plus d'une minute:

apiVersion: monitoring.coreos.com/v1

kind: PrometheusRule

metadata:

name: backend-cpu-alert

namespace: observ-prod

labels:

release: monitoring

spec:

groups:

- name: backend.rules

rules:

- alert: BackendCpuLimitWarning

expr: >

```
sum(rate(container_cpu_usage_seconds_total{namespace="backend",
container="backend"}[1m]))
```

/

```
sum(kube_pod_container_resource_limits{namespace="backend", container="backend",
resource="cpu"})
```

> 0.80

for: 1m

labels:

severity: warning

annotations:

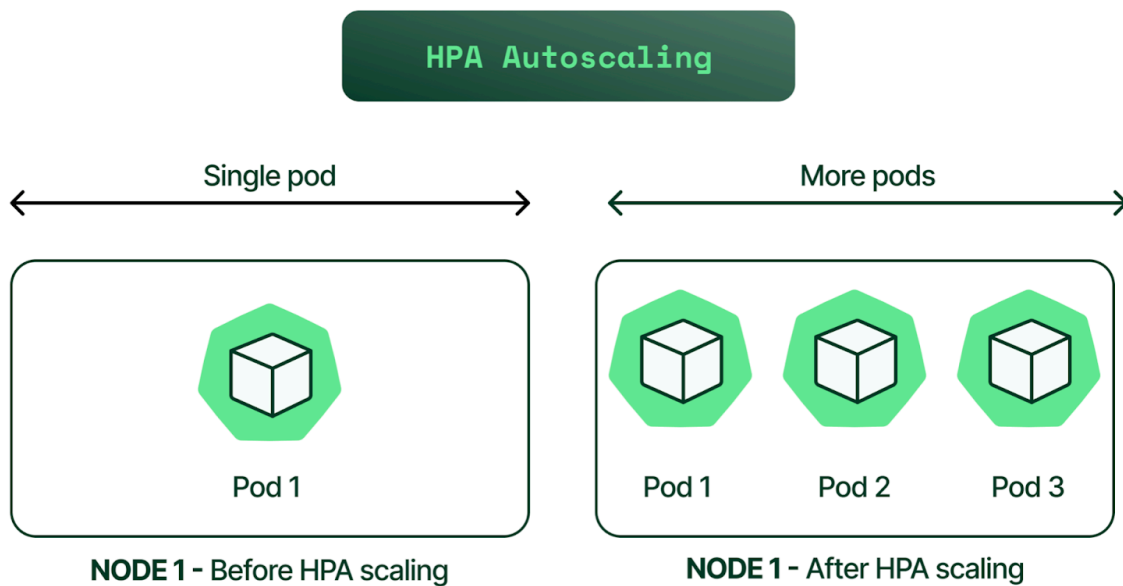
summary: "Alerte : Le Backend frôle sa limite CPU"

description: "Le pod backend dans le namespace backend consomme plus de 80% de sa limite CPU maximale allouée depuis plus d'une minute."

Note: Il peut être judicieux de setup l'envoi par mail d'alertes.

Maintenant, nous allons apprendre qu'en cas de surcharge CPU, le cluster puisse "scale". C'est-à-dire ajouter un ou plusieurs pods en réaction. Il y a le concept d'horizontal pod autoscale:

<https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>



“Horizontal scaling means that the response to increased load is to deploy more Pods. This is different from vertical scaling, which for Kubernetes would mean assigning more resources (for example: memory or CPU) to the Pods that are already running for the workload.

If the load decreases, and the number of Pods is above the configured minimum, the HorizontalPodAutoscaler instructs the workload resource (the Deployment, StatefulSet, or other similar resource) to scale back down.

Kubernetes implements horizontal pod autoscaling as a control loop that runs intermittently (it is not a continuous process). The interval is set by the `--horizontal-pod-autoscaler-sync-period` parameter to the `kube-controller-manager` (and the default interval is 15 seconds).

The common use for HorizontalPodAutoscaler is to configure it to fetch metrics from aggregated APIs (`metrics.k8s.io`, `custom.metrics.k8s.io`, or `external.metrics.k8s.io`). The `metrics.k8s.io` API is usually provided by an add-on named Metrics Server, which needs to be launched separately”

Nous avons installé le metric server au tout début.

<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: backend
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 1
  maxReplicas: 3
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80

```

On indique ci-dessus qu'on peut scale jusqu'à 3 replicas, et la target cible est 80% d'usage cpu moyen.

```

etudiant@debian:~/projet_kubernetes/backend$ kubectl get hpa -n backend

```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
backend-hpa	Deployment/backend	cpu: 1%/80%	1	3	1	23s

On le bombarde de requêtes pour voir le scaling en oeuvre

```

etudiant@debian:~/projet_kubernetes/frontend$ kubectl run -i --tty load-generator --rm --image=busybox
--restart=Never --labels="app=frontend" -n frontend -- /bin/sh -c "while true; do wget -q -O- http://
backend-svc.backend.svc.cluster.local:3000/products > /dev/null; done"
All commands and output from this session will be recorded in container logs, including credentials an
d sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.

```

```

etudiant@debian:~/projet_kubernetes/grafana$ kubectl get hpa -n backend -w

```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
backend-hpa	Deployment/backend	cpu: 1%/80%	1	3	1	4m
backend-hpa	Deployment/backend	cpu: 38%/80%	1	3	1	6m1s
backend-hpa	Deployment/backend	cpu: 72%/80%	1	3	1	6m16s
backend-hpa	Deployment/backend	cpu: 160%/80%	1	3	1	6m31s
backend-hpa	Deployment/backend	cpu: 98%/80%	1	3	2	6m46s
backend-hpa	Deployment/backend	cpu: 47%/80%	1	3	2	7m1s

On constate le passage d'1 replica à 2 replicas.

On peut le voir via la commande get:

```
etudiant@debian:~$ kubectl get pods -n backend
NAME                                READY   STATUS    RESTARTS   AGE
backend-7786458c8c-fmgpf           1/1     Running   0           37s
backend-7786458c8c-l7h5z           1/1     Running   0           2d6h
```

5 minutes après ça scale down:

```
etudiant@debian:~/projet_kubernetes/grafana$ kubectl get hpa -n backend -w
NAME           REFERENCE             TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa    Deployment/backend     cpu: 1%/80%      1          3          2           9m18s
backend-hpa    Deployment/backend     cpu: 0%/80%      1          3          2           10m
backend-hpa    Deployment/backend     cpu: 1%/80%      1          3          2           10m
backend-hpa    Deployment/backend     cpu: 0%/80%      1          3          2           10m
backend-hpa    Deployment/backend     cpu: 1%/80%      1          3          2           11m
backend-hpa    Deployment/backend     cpu: 0%/80%      1          3          2           11m
backend-hpa    Deployment/backend     cpu: 1%/80%      1          3          2           11m
backend-hpa    Deployment/backend     cpu: 0%/80%      1          3          2           12m
backend-hpa    Deployment/backend     cpu: 1%/80%      1          3          2           12m
backend-hpa    Deployment/backend     cpu: 0%/80%      1          3          2           13m
backend-hpa    Deployment/backend     cpu: 0%/80%      1          3          2           13m
backend-hpa    Deployment/backend     cpu: 1%/80%      1          3          1           13m
```

Phase 8 : Automatisation & Résilience

On installe le cert-manager pour chiffrer le trafic du site web shop.local.

```
etudiant@debian:~/projet_kubernetes/frontend$ helm repo add jetstack https://charts.jetstack.io
helm repo update
```

```
etudiant@debian:~/projet_kubernetes/frontend$ helm install cert-manager jetstack/cert-manager \
--namespace cert-manager \
--create-namespace \
--set crds.enabled=true
NAME: cert-manager
LAST DEPLOYED: Tue Mar 10 16:16:33 2026
NAMESPACE: cert-manager
STATUS: deployed
```

Cela a créé le namespace cert-manager.

<https://cert-manager.io/docs/concepts/issuer/>

<https://cert-manager.io/docs/configuration/selfsigned/>

On va l'auto signé.

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}
```

```
etudiant@debian:~/projet_kubernetes/cert-manager$ kubectl get clusterissuers.cert-manager.io
NAME                READY   AGE
selfsigned-issuer   True    34s
```

On modifie alors l'ingress du frontend et on spécifie cert-manager.io:

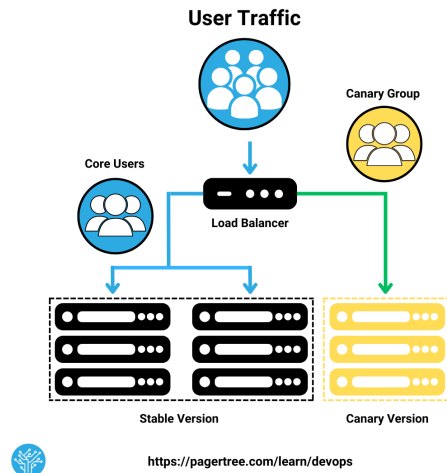
```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
  namespace: frontend
  annotations:
    cert-manager.io/cluster-issuer: "selfsigned-issuer"
spec:
  ingressClassName: traefik
  rules:
  - host: shop.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
  tls:
  - hosts:
    - shop.local
    secretName: shop-tls
```

Mais puisque ce n'est pas une CA connue, alors on obtient:

```
etudiant@debian2:~$ curl https://shop.local/products
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the webpage mentioned above.
```

Nous allons passer au déploiement canary.



Un déploiement canary consiste à faire transiter une partie du trafic vers une version non stable, afin de récolter des informations dessus et derrière pouvoir ensuite transiter 100% du trafic s'il n'y a pas d'erreurs.

On installe argo-rollouts pour les déploiement canary:

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops$ kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-rollouts/releases/latest/download/install.yaml
namespace/argo-rollouts created
```

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/argo$ curl -LO https://github.com/argoproj
/argo-rollouts/releases/latest/download/kubectl-argo-rollouts-linux-amd64
chmod +x ./kubectl-argo-rollouts-linux-amd64
sudo mv ./kubectl-argo-rollouts-linux-amd64 /usr/local/bin/kubectl-argo-rollouts

```

% Total	% Received	% Xferd	Average	Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left	Speed
0	0	0	0	0	--:--:--	--:--:--	--:--:--	0
0	0	0	0	0	--:--:--	--:--:--	--:--:--	0
100	123M	100	123M	0	0	56.8M	0	0:00:02 0:00:02 --:--:-- 73.6M

```

[sudo] password for etudiant:
etudiant@debian:~/projet_cloud_security/cloud_native_devops/argo$ kubectl argo rollouts version
kubectl-argo-rollouts: v1.8.4+55e89dd
  BuildDate: 2026-02-13T22:01:42Z
  GitCommit: 55e89ddc5329db9ebd72edff5503f2822d2e80ff
  GitTreeState: clean
  GoVersion: go1.23.12
  Compiler: gc
  Platform: linux/amd64

```

On modifie le frontend en canary release (20% du traffic)

```

apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: frontend
  namespace: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {}
  template:
    metadata:
      labels:
        app: frontend
    spec:
      serviceAccountName: frontend-sa
      containers:
        - name: frontend
          image: 192.168.1.61:5000/frontend:v1.1
          imagePullPolicy: Always
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "250m"
              memory: "256Mi"
          ports:
            - containerPort: 8080

```

Pause permettra d'attendre validation humaine avant de faire transférer encore plus de traffic.

On peut par exemple faire 20%/50%/70% et avant chaque étape il faudra saisir une commande pour valider le passage à plus de traffic.

On delete l'ancien deployment avant d'appliquer


```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/manifests$ kubectl delete deploym
ent frontend -n frontend
deployment.apps "frontend" deleted from frontend namespace

```

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/manifests$ kubectl argo rollouts
get rollout frontend -n frontend
Name:          frontend
Namespace:     frontend
Status:        ✓Healthy
Strategy:      Canary
  Step:        2/2
  SetWeight:   100
  ActualWeight: 100
Images:        192.168.1.61:5000/frontend:v1.1 (stable)
Replicas:
  Desired:     1
  Current:     1
  Updated:     1
  Ready:       1
  Available:   1

```

NAME	KIND	STATUS	AGE	INFO
○ frontend	Rollout	✓Healthy	6m10s	
└─# revision:1				
└─┐ frontend-58cc94d765	ReplicaSet	✓Healthy	6m10s	stable
└─┐ frontend-58cc94d765-2jv1w	Pod	✓Running	6m10s	ready:1/1

Pour l'instant il n'y a qu'un pod qui reçoit tout le trafic.

Je modifie alors la version de l'image de v1.1 à v1.20

```

containers:
- name: frontend
  image: 192.168.1.61:5000/frontend:v1.20

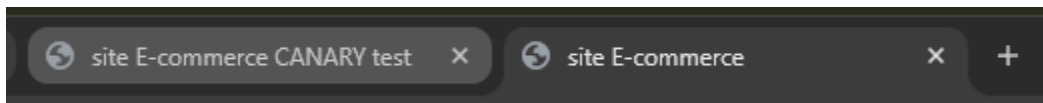
```

J'apply. Comme on peut le voir c'est en pause volontairement.

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/manifests$ kubectl argo rollouts
get rollout frontend -n frontend
Name:          frontend
Namespace:     frontend
Status:        II Paused
Message:       CanaryPauseStep
Strategy:      Canary
  Step:        1/2
  SetWeight:   20
  ActualWeight: 50
Images:        192.168.1.61:5000/frontend:v1.1 (stable)
               192.168.1.61:5000/frontend:v1.20 (canary)
Replicas:
  Desired:     1
  Current:     2
  Updated:     1
  Ready:       2
  Available:   2

NAME                                KIND      STATUS      AGE      INFO
○ frontend                          Rollout    II Paused   18m
├─# revision:2
│   └─ frontend-5d6c5688cb           ReplicaSet ✓Healthy    2m27s    canary
│       └─ frontend-5d6c5688cb-vchb7 Pod        ✓Running    2m27s    ready:1/1
└─# revision:1
    └─ frontend-58cc94d765           ReplicaSet ✓Healthy    18m      stable
        └─ frontend-58cc94d765-2jv1w Pod        ✓Running    18m      ready:1/1
```

J'ai modifié la balise <title> sur la version canary. Ainsi en actualisation plusieurs fois, on a 1 chance sur 5 d'avoir CANARY test sur le <title> du site.



Après vérification des métriques tel que le cpu, etc. on veut valider la release et “promote” pour que 100% du trafic aille sur la nouvelle version.

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/manifests$ kubectl argo rollouts
promote frontend -n frontend
rollout 'frontend' promoted
```

Après promotion, dans 100% des rafraichissement de la page la balise <title> contient CANARY test. C'est donc fonctionnel.

On peut ensuite ajouter Kustomize à notre cluster.

<https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

Kustomize permettra d'avoir un environnement prod et staging. Pour ce faire il utilise un fichier kustomization.yaml regroupant toutes les ressources associés.

Kustomize utilise le dossier base pour y stocker les yaml selon l'environnement, et overlays/prod ou overlay/staging pour déployer ses yaml:

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ mkdir -p base overlays/prod
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ ls
app base manifests overlays
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ mv manifests/* base/
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ ls base/
default-policy.yaml deployment.yaml full-policy.yaml serviceaccount.yaml
deployment.old frontend-svc.yaml ingress.yaml
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ ls manifests/
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ rmdir manifests
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$
```

Dans le dossier base, kustomization.yaml, on indique nos resources:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml
- frontend-svc.yaml
- ingress.yaml
- serviceaccount.yaml
- default-policy.yaml
- full-policy.yaml
```

Dans le dossier overlays/prod:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

namespace: frontend
```

Nous avons fait de même pour backend et database qui ont leur dossier base et overlays.

Le problème de pause : {} est que ce n'est pas automatisé. On va donc utiliser le kind AnalysisTemplate pour déterminer un critère selon lequel on considère la canary release

comme déployable. Nous allons considérer que c'est un succès lorsque la charge cpu est inférieure à 100m (la limite définie).

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: cpu-health-check
spec:
  metrics:
  - name: check-cpu-usage
    count: 2
    interval: 30s
    successCondition: result < 0.1 #inférieur à 100m
    provider:
      prometheus:
        address: http://monitoring-kube-prometheus-prometheus.observe-prod.svc.cluster.local:9090
        query: |
          scalar(sum(rate(container_cpu_usage_seconds_total{namespace="frontend", pod=~"frontend-[0-9]"
```

On modifie alors notre rollout:

```
strategy:
  canary:
    steps:
    - setWeight: 20
    - analysis:
        templates:
        - templateName: cpu-health-check
    - setWeight: 100
```

On n'oublie pas de le rajouter au kustomization.yaml de base:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml
- frontend-svc.yaml
- ingress.yaml
- serviceaccount.yaml
- default-policy.yaml
- full-policy.yaml
- frontend-analysis.yaml
```

On peut ensuite vérifier que tout est bon

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops$ kubectl kustomize frontend/overlays/prod
apiVersion: v1
automountServiceAccountToken: false
kind: ServiceAccount
metadata:
  name: frontend-sa
  namespace: frontend
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
  namespace: frontend
```

On apply:

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops$ kubectl apply -k frontend/overlays/prod
serviceaccount/frontend-sa unchanged
service/frontend-svc unchanged
analysisplatform.argoproj.io/cpu-health-check created
rollout.argoproj.io/frontend configured
ingress.networking.k8s.io/frontend-ingress unchanged
networkpolicy.networking.k8s.io/allow-traefik unchanged
networkpolicy.networking.k8s.io/default-deny-all unchanged
```

On modifie le customization.yaml dans prod:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

namespace: frontend

images:
- name: 192.168.1.61:5000/frontend
  newTag: v1.22
```

Et on apply:

```

Name:      frontend
Namespace: frontend
Status:    ✓Healthy
Strategy:  Canary
  Step:    3/3
  SetWeight: 100
  ActualWeight: 100
Images:    192.168.1.61:5000/frontend:v1.22 (stable)
Replicas:
  Desired: 1
  Current: 1
  Updated: 1
  Ready:   1
  Available: 1

```

NAME	KIND	STATUS	AGE	INFO
frontend	Rollout	✓Healthy	3h17m	
# revision:3				
frontend-6f7d784886	ReplicaSet	✓Healthy	8m31s	stable
frontend-6f7d784886-4mmm	Pod	✓Running	60s	ready:1/1
frontend-6f7d784886-3-1	AnalysisRun	⚠Error	8m30s	⚠5
frontend-6f7d784886-3-1.1	AnalysisRun	✓Successful	59s	✓2
# revision:2				
frontend-5f87fb857c	ReplicaSet	• ScaledDown	3h15m	
# revision:1				
frontend-5d6c5688cb	ReplicaSet	• ScaledDown	3h17m	

Il y avait une petite faute de syntaxe, donc il y a eu une erreur il y a 8m30s. Le comportement d'argo rollout a alors été de laisser le trafic vers la version stable. Maintenant que tout marche, on constate que l'actuel weight est à 100, et que l'ancienne version (revision 2) a scale down. Tout est bon.

Pour avoir un environnement "pré-prod"(staging), il suffit simplement de créer un dossier staging dans overlays, d'y associer le kustomization.

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/overlays$ ls
prod staging
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/overlays$ ls staging/
kustomization.yaml

```

Ci-dessous on peut considérer que l'environnement staging utilisera une image différente:

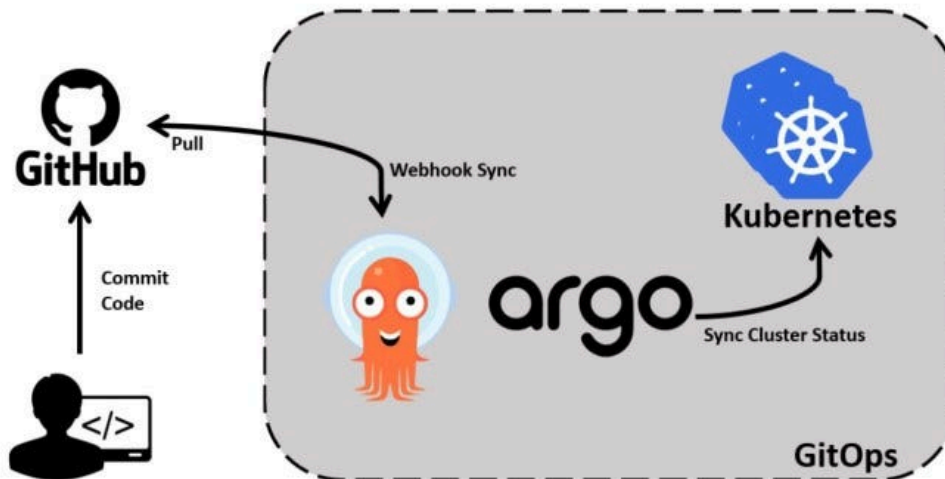
```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
namespace: frontend-staging
images:
- name: 192.168.1.61:5000/frontend
  newTag: betav1.3

```

On pourra alors utiliser l'environnement de frontend pour le déployer dans un tout nouveau namespace intitulé frontend-staging, avec une autre version de l'image pour tester. Ainsi l'environnement prod peut tourner indépendamment. Il est bien entendu possible de faire cela avec le backend, etc. Il pourra être judicieux de créer une branche appropriée sur GitHub pour cet environnement.

Pour la suite, nous utiliserons argoCD. ArgoCD va permettre de synchroniser notre cluster et GIT. L'état du GIT sera l'état de notre cluster.



On installe argocd:

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f
```

```
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

<https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/>

On crée les projets pour chaque namespace:

```

apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: prod
  namespace: argocd
spec:
  description: "environnement prod complet"
  sourceRepos:
    - "git@github.com:YassineDerouich/projet_cloud_security.git"

  destinations:
    - namespace: frontend
      server: https://kubernetes.default.svc
    - namespace: backend
      server: https://kubernetes.default.svc
    - namespace: database
      server: https://kubernetes.default.svc

  namespaceResourceWhitelist:
    - group: 'apps'
      kind: Deployment
    - group: 'apps'
      kind: StatefulSet
    - group: 'argoproj.io'
      kind: Rollout
    - group: 'argoproj.io'
      kind: AnalysisTemplate
    - group: ''
      kind: Service
    - group: ''
      kind: ConfigMap
    - group: ''
      kind: Secret
    - group: ''
      kind: ServiceAccount
    - group: ''
      kind: PersistentVolumeClaim
    - group: 'networking.k8s.io'
      kind: Ingress
    - group: 'networking.k8s.io'
      kind: NetworkPolicy
    - group: 'autoscaling'
      kind: HorizontalPodAutoscaler
    - group: 'external-secrets.io'
      kind: ExternalSecret

  clusterResourceWhitelist: []

```


On a spécifié les ressources possibles à créer au niveau du namespace. Et il est impossible de déborder sur le cluster avec la dernière ligne. Ceci est bon pour la sécurité.

On spécifie ensuite , pour frontend/backend/database, où se trouve l'application en question dans notre repository GitHub:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: frontend-prod
  namespace: argocd
spec:
  project: prod
  source:
    repoURL: 'git@github.com:YassineDerouich/projet_cloud_security.git'
    targetRevision: main
    path: cloud_native_devops/frontend/overlays/prod
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: frontend
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Prune:true permet de supprimer les ressources présentes dans le cluster mais qui ont disparu dans le repo GitHub.

selfHeal:true , si une personne modifie le cluster en direct et que cela diffère du repo git alors argocd conservera la version GIT comme source de vérité

Note: il faudra faire de même pour le backend et le db. Qui auront leur propre "application" argocd. Les push seront indépendants.

L'objectif final sera d'avoir :

 **backend-prod**  

Project: prod

Labels:

Status:  Healthy  Synced

Repository: git@github.com:YassineDerouich/projet_cloud_security.git

Target Revision: main

Path: cloud_native_devops/backend/overlays/prod

Destination: in-cluster

Namespace: backend

Created At: 03/12/2026 14:11:23 (44 minutes ago)

Last Sync: 03/12/2026 14:53:11 (2 minutes ago)

 SYNC  

 **database-prod**  

Project: prod

Labels:

Status:  Healthy  Synced

Repository: git@github.com:YassineDerouich/projet_cloud_security.git

Target Revision: main

Path: cloud_native_devops/database/overlays/prod

Destination: in-cluster

Namespace: database

Created At: 03/12/2026 14:11:27 (43 minutes ago)

Last Sync: 03/12/2026 14:49:08 (6 minutes ago)

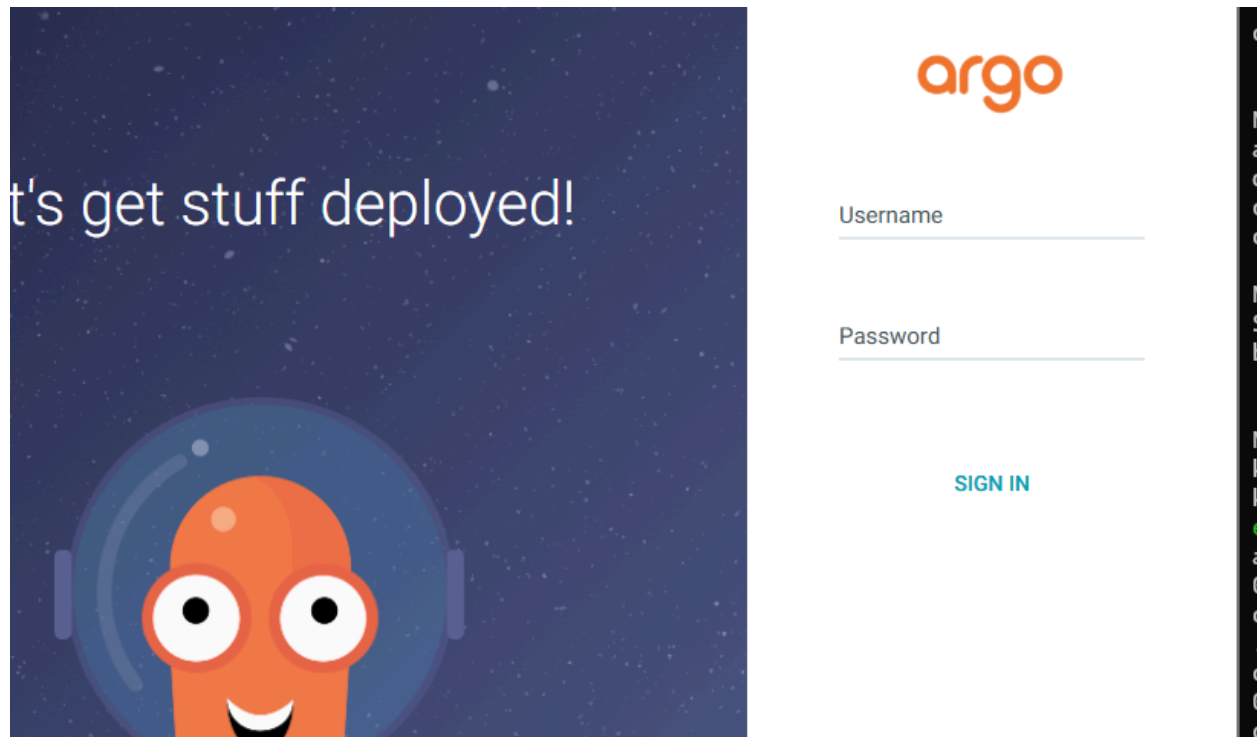
 SYNC  

On a ceci. On apply pas encore, on doit configurer Github.

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/argo$ ls
frontend-app.yaml  project-prod.yaml
```

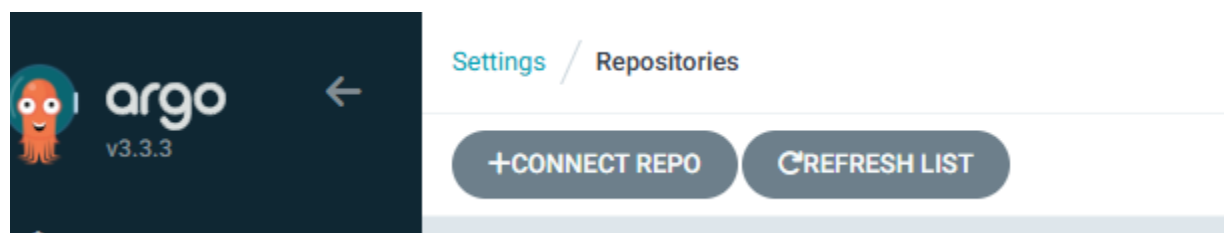
Par défaut il listen en 127.0.0.1, corrigeons cela pour y accéder mon navigateur:

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/argo$ kubectl port-forward svc/argocd-server -n argocd 8080:443 --address 0.0.0.0
```



On récupère le password via `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d && echo`
Qu'on modifiera plus tard.

Il faut ensuite se rendre dans settings/repositories



Et connect repo:

CONNECT

SAVE AS CREDENTIALS TEMPLATE

CANCEL

Choose your connection method:

VIA SSH ▼

CONNECT REPO USING SSH

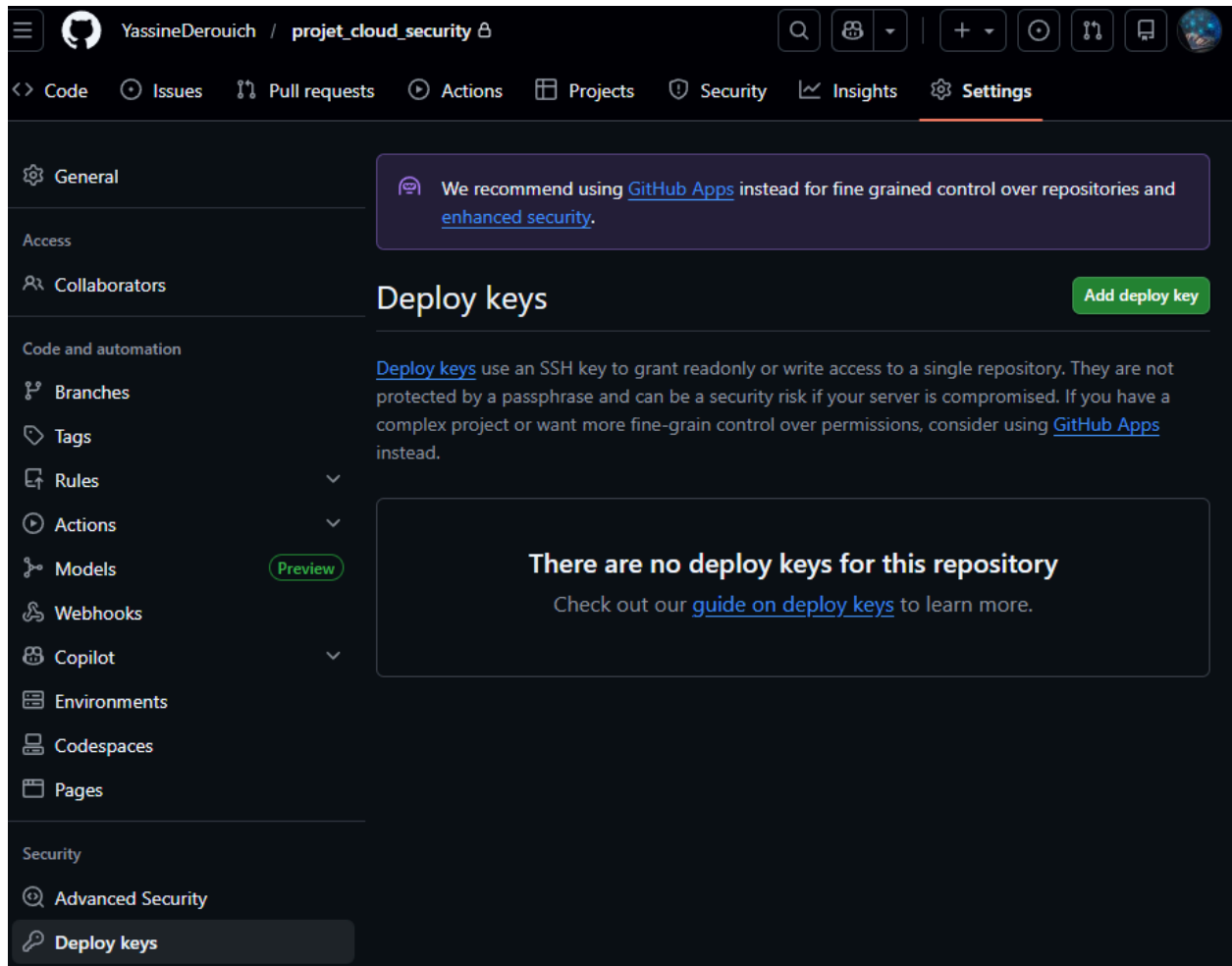
Name (mandatory for Helm)

Project

Repository URL

SSH private key data

Il faut d'abord créer la clé ssh via github.



On génère la clé via `ssh-keygen -t ed25519 -C "argocd-deploy-key" -f ./id_ed25519_argo -N ""`

On saisit la clé publique sur github (le contenu de `cat id_ed25519_argo.pub`)

Ensuite via `cat id_ed25519_argo` on la copie et la colle sur github.

Choose your connection method:

VIA SSH ▼

CONNECT REPO USING SSH

Name (mandatory for Helm)

prod

Project

prod

Repository URL

git@github.com:YassineDerouich/projet_cloud_security.git

SSH private key data

-----BEGIN OPENSSH PRIVATE KEY-----

A large black rectangular redaction box covering the SSH private key content.

-----END OPENSSH PRIVATE KEY-----

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/argo$ kubectl apply -f project-prod.yaml
appproject.argoproj.io/prod created
etudiant@debian:~/projet_cloud_security/cloud_native_devops/argo$ kubectl apply -f frontend-app.yaml
application.argoproj.io/frontend-prod created
```

 **frontend-prod**

Project:	prod
Labels:	
Status:	♥ Healthy ✓ Synced
Repository:	git@github.com:YassineDerouich/projet_cloud_security.git
Target Revision:	main
Path:	cloud_native_devops/frontend/overlays/prod
Destination:	in-cluster
Namespace:	frontend
Created At:	03/12/2026 10:56:35 (a few seconds ago)
Last Sync:	03/12/2026 10:56:37 (a few seconds ago)

 SYNC  

Allons sur le site shop.local avant modif:



welcome

Vault and Kubernetes.

[Home](#) | [About](#) | [Products](#)

Etat du kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

namespace: frontend
```

Modifions le html:

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/app/html$ cat index.html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>site E-commerce BETA STAGING</title>
  <style>body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; }</style>
</head>
<body>
  <h1>welcome</h1>
  <p>ARGOCD test Vault and Kubernetes.</p>
  <nav>
    <a href="/">Home</a> | <a href="/about">About</a> | <a href="/products">Products</a>
  </nav>
  <div id="api-status" style="margin-top: 20px; color: gray;">
    Backend is coming
  </div>
</body>
</html>
```

On crée la nouvelle image avec dockerhub, on met à jour le yaml correspond et on apply.

```
containers:
- name: frontend
  image: 192.168.1.61:5000/frontend:v1.31
```

On constate que le pod a immédiatement été "scaledown". Autrement dit, ArgoCD ne l'a pas laissé être la version stable car je ne l'ai pas encore push sur GitHub.

NAME	KIND	STATUS	AGE	INFO
frontend	Rollout	✓Healthy	24h	
└─# revision:5				
└─└─frontend-6f7d784886	ReplicaSet	✓Healthy	21h	stable
└─└─└─frontend-6f7d784886-4mmmn	Pod	✓Running	21h	ready:1/1
└─# revision:4				
└─└─frontend-78f78b5995	ReplicaSet	• ScaledDown	3m35s	

Pushons et observons.

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops$ git add .
etudiant@debian:~/projet_cloud_security/cloud_native_devops$ git commit -m "image frontend v1.31"
[main 88c252b] image frontend v1.31
 3 files changed, 2 insertions(+), 6 deletions(-)
etudiant@debian:~/projet_cloud_security/cloud_native_devops$ git push
```



```

etudiant@debian:~/projet_cloud_security/cloud_native_devops$ kubectl argo rollouts get rollout frontend -n frontend
Name:          frontend
Namespace:     frontend
Status:        ✓Healthy
Strategy:      Canary
  Step:        3/3
  SetWeight:   100
  ActualWeight: 100
Images:        192.168.1.61:5000/frontend:v1.31 (stable)
Replicas:
  Desired:     1
  Current:     1
  Updated:     1
  Ready:       1
  Available:   1

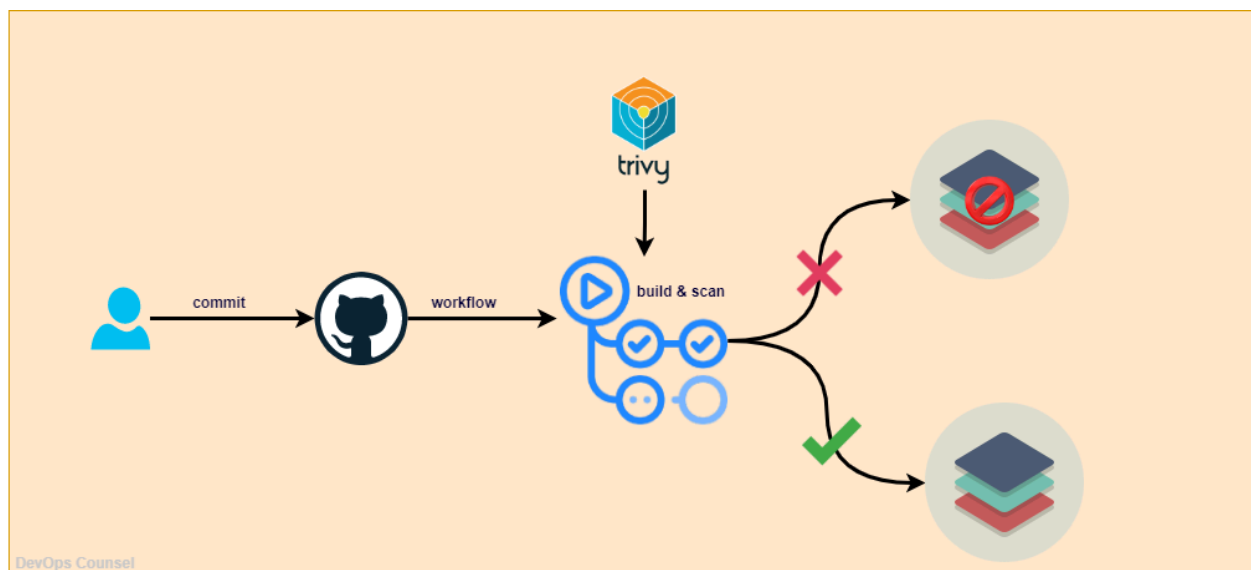
NAME          KIND      STATUS      AGE   INFO
○ frontend    Rollout   ✓Healthy    24h
├─ # revision:12
│   └─ frontend-6784964f7d
│       ├── frontend-6784964f7d-6dghr Pod      ✓Healthy    2m9s  stable
│       └─ frontend-6784964f7d-12-1 AnalysisRun ✓Running    36s   ready:1/1
└─ frontend-6784964f7d-12-1 AnalysisRun ✓Successful 35s   ✓2

```

Après avoir synchronisé ArgoCD, c'est bon la dernière version est la bonne.

Phase 9 : L'Audit Continu (SecOps & Conformité)

On installe trivy via Helm (C'est un scanner de vulnérabilités.)



Trivy est puissant et permet par exemple d'auditer nos images, ainsi que nos manifests kubernetes.

Installons via helm:

```
helm repo add aqua https://aquasecurity.github.io/helm-charts/
```

```
helm repo update
```

```
helm install trivy-operator aqua/trivy-operator \ --namespace security \  
--set="trivy.ignoreUnfixed=true"
```

Le paramètre ignoreUnfixed=true permet de ne remonter que les vulnérabilités qui ont un correctif disponible, ça évite de polluer les rapports avec des failles qu'on ne peut de toute façon pas corriger pour l'instant.

<https://oneuptime.com/blog/post/2026-02-02-trivy-kubernetes-scanning/view>

On saisit ensuite : `kubectl get vulnerabilityreports -n frontend -o json`

Permet de lister toutes les vulnérabilités au sein du namespace frontend

Ainsi il a trouvé une CVE critique sur l'image alpine utilisé par mon frontend:

```
{  
  "summary": {  
    "criticalCount": 1,  
    "highCount": 0,  
    "lowCount": 0,  
    "mediumCount": 1,  
    "noneCount": 0,  
    "unknownCount": 0  
  },  
  "updateTimestamp": "2026-03-12T10:46:14Z",  
  "vulnerabilities": [  
    {  
      "fixedVersion": "1.3.2-r0",  
      "installedVersion": "1.3.1-r2",  
      "lastModifiedDate": "2026-01-15T14:16:27Z",  
      "links": [],  
      "packagePURL": "pkg:apk/alpine/zlib@1.3.1-r2?arch=x86_64\u0026distro=3.23.3",  
      "primaryLink": "https://avd.aquasec.com/nvd/cve-2026-22184",  
      "publishedDate": "2026-01-07T21:16:01Z",  
      "resource": "zlib",  
      "score": 8.6,  
      "severity": "CRITICAL",  
      "target": "",  
      "title": "zlib: zlib: Arbitrary code execution via buffer overflow in untgz utility",  
      "vulnerabilityID": "CVE-2026-22184"  
    }  
  ]  
}
```

En regardant de plus près, on apprend la version à installer pour résoudre cela:

```
{  
  "fixedVersion": "1.3.2-r0",  
  "installedVersion": "1.3.1-r2",  
}
```

On constate bien que j'ai une ancienne version 1.3.1

Concernant le backend, il n'y a aucune vulnérabilité (patchable):

```
repository: backend
tag: v1.9
os:
  family: debian
  name: "13.3"
registry:
  server: 192.168.1.61:5000
scanner:
  name: Trivy
  vendor: Aqua Security
  version: 0.69.1
summary:
  criticalCount: 0
  highCount: 0
  lowCount: 0
  mediumCount: 0
  noneCount: 0
```

Observons désormais nos manifests kubernetes pour constater s'il y a des risques de sécurité.

kubectl get configauditreports -n frontend -o json

Le résultat est énorme car Il vérifie pour chaque ressource (hors CRD):

```

    "name": "networkpolicy-allow-traefik",
    "namespace": "frontend",
    "ownerReferences": [
      {
        "apiVersion": "networking.k8s.io/v1",
        "blockOwnerDeletion": false,
        "controller": true,
        "kind": "NetworkPolicy",
        "name": "allow-traefik",
        "uid": "a4380409-6d75-48e0-81eb-1528326dbebc"
      }
    ],
    "resourceVersion": "578946",
    "uid": "952fff8c-c8f5-4457-98e8-9fbfc179835c"
  },
  "port": {
    "checks": [],
    "scanner": {
      "name": "Trivy",
      "vendor": "Aqua Security",
      "version": "0.30.0"
    },
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    }
  }
}

```

Via un grep on peut savoir plus facilement;

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ kubectl get configauditreports -n
frontend -o json | grep summary -A 5
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 3,
      "lowCount": 6,
      "mediumCount": 4
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },

```

On constate que c'est la 4ème ressource analysée qui pose problème. Via jq (une sorte de sed pour le json), on obtient davantage d'informations:

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend$ kubectl get configauditreports -n
frontend -o json | jq '.items[].report.checks[] | select(.severity == "HIGH") | {title: .title, message:
e: .messages[0], description: .description}'
{
  "title": "Root file system is not read-only",
  "message": "Container 'frontend' of ReplicaSet 'frontend-6784964f7d' should set 'securityContext.read
OnlyRootFilesystem' to true",
  "description": "An immutable root file system prevents applications from writing to their local disk.
This can limit intrusions, as attackers will not be able to tamper with the file system or write forei
gn executables to disk."
}
{
  "title": "Default security context configured",
  "message": "container frontend-6784964f7d in frontend namespace is using the default security context
",
  "description": "Security context controls the allocation of security parameters for the pod/container
/volume, ensuring the appropriate level of protection. Relying on default security context may expose v
ulnerabilities to potential attacks that rely on privileged access."
}
{
  "title": "Default security context configured",
  "message": "replicaset frontend-6784964f7d in frontend namespace is using the default security contex
t, which allows root privileges",
  "description": "Security context controls the allocation of security parameters for the pod/container
/volume, ensuring the appropriate level of protection. Relying on default security context may expose v
ulnerabilities to potential attacks that rely on privileged access."
}

```

Pour rappel l'image du frontend:

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/app$ cat Dockerfile
FROM nginxinc/nginx-unprivileged:alpine-slim

COPY default.conf /etc/nginx/conf.d/default.conf

COPY ./html /usr/share/nginx/html

USER 101

EXPOSE 8080

CMD ["nginx", "-g", "daemon off;"]
```

“The nginx-unprivileged creates a nginx user with gid and uid 101”

La première correction de vulnérabilité consiste à mettre sur True l'option `readOnlyRootFilesystem`. Elle indique qu'il ne sera possible que d'avoir les permissions de lecture sur le système de fichiers racine de l'image.

Or, le frontend a besoin d'écrire des logs. Nginx recommande alors de créer alors de monter les volumes.

Concernant le security context, nous allons spécifier de run l'image en tant qu'user 101 et `allowPrivilegeEscalation:False` empêchera à un processus de gagner plus de droits qu'au départ.

Cf. <https://docs.nginx.com/waf/configure/kubernetes-read-only/>

Version modifié du deployment:

```
template:
  metadata:
    labels:
      app: frontend
  spec:
    serviceAccountName: frontend-sa
    securityContext:
      runAsNonRoot: true
      runAsUser: 101
```

```
containers:
- name: frontend
  image: 192.168.1.61:5000/frontend:v1.31
  imagePullPolicy: Always
  securityContext:
    readOnlyRootFilesystem: true
    allowPrivilegeEscalation: false
```

```
ports:
- containerPort: 8080

volumeMounts:
- mountPath: /var/cache/nginx
  name: nginx-cache
- mountPath: /var/run
  name: nginx-run
- mountPath: /tmp
  name: tmp-volume

volumes:
- name: nginx-cache
  emptyDir: {}
- name: nginx-run
  emptyDir: {}
- name: tmp-volume
  emptyDir: {}
```

On git push.

LAST SYNC ●

⋮


Sync OK

Succeeded a minute ago

Author: Yassine Derouich <97849927+Yas...>
 Comment: fix deployment frontend

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/base$ kubectl argo rollouts get ro
llouts frontend -n frontend
Name:          frontend
Namespace:     frontend
Status:        ◯ Progressing
Message:       old replicas are pending termination
Strategy:      Canary
  Step:        1/3
  SetWeight:   20
  ActualWeight: 50
Images:        192.168.1.61:5000/frontend:v1.31 (canary, stable)
Replicas:
  Desired:     1
  Current:     2
  Updated:     1
  Ready:       2
  Available:   2

NAME                                KIND      STATUS      AGE   INFO
○ frontend                          Rollout    ◯ Progressing 26h
├─# revision:14
│   └─ frontend-858c9ffb56           ReplicaSet ✓ Healthy   13s   canary
│       └─ frontend-858c9ffb56-ffvnx Pod        ✓ Running   13s   ready:1/1
│           └─ frontend-858c9ffb56-14-1 AnalysisRun ◯ Running   13s   ✓1

```

Nous avons alors la nouvelle version du rollout.

On delete l'ancien rapport d'audit de conf:

```
kubectl delete configauditreports --all -A
```

Par défaut, il fait un rapport toutes les 6h.

Nous n'avons plus aucune erreur de configuration de sécurité de niveau high.


```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/frontend/base$ kubectl get configauditreports -n frontend -o json | grep summary -A 5
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 6,
      "mediumCount": 2
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
}

```

Nous allons appliquer les mêmes changements pour le backend.

Pour rappel l'image du backend:

```

etudiant@debian:~/projet_cloud_security/cloud_native_devops/backend/app$ cat Dockerfile
FROM node:24-alpine AS builder

WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .

FROM gcr.io/distroless/nodejs24-debian13

ENV NODE_ENV=production

WORKDIR /usr/src/app

COPY --from=builder --chown=nonroot:nonroot /usr/src/app /usr/src/app

EXPOSE 3000

CMD ["server.js"]

```

L'image est déjà sécurisée mais appliquons malgré tout les modifications, sauf les volumes et pas d'user 101.

On delete les anciens configauditreport, et on constate qu'il n'y a plus de vulnérabilité "high":

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/backend/manifests$ kubectl get configauditreports -n backend -o json | grep summary -A 5
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 6,
      "mediumCount": 2
    },
--
    "summary": {
      "criticalCount": 0,
      "highCount": 0,
      "lowCount": 0,
      "mediumCount": 0
    },
--
```

Concernant la database postgres nous n'avons qu'une vulnérabilité high:

```
--
    "summary": {
      "criticalCount": 0,
      "highCount": 1,
      "lowCount": 10,
      "mediumCount": 2
    },
--
```

```
etudiant@debian:~/projet_cloud_security/cloud_native_devops/backend/manifests$ kubectl get configauditreports -n database -o json | jq '.items[].report.checks[] | select(.severity == "HIGH") | {title: .title, message: .messages[0], description: .description}'
[
  {
    "title": "Root file system is not read-only",
    "message": "Container 'postgres' of StatefulSet 'postgres' should set 'securityContext.readOnlyRootFilesystem' to true",
    "description": "An immutable root file system prevents applications from writing to their local disk. This can limit intrusions, as attackers will not be able to tamper with the file system or write foreign executables to disk."
  }
]
```

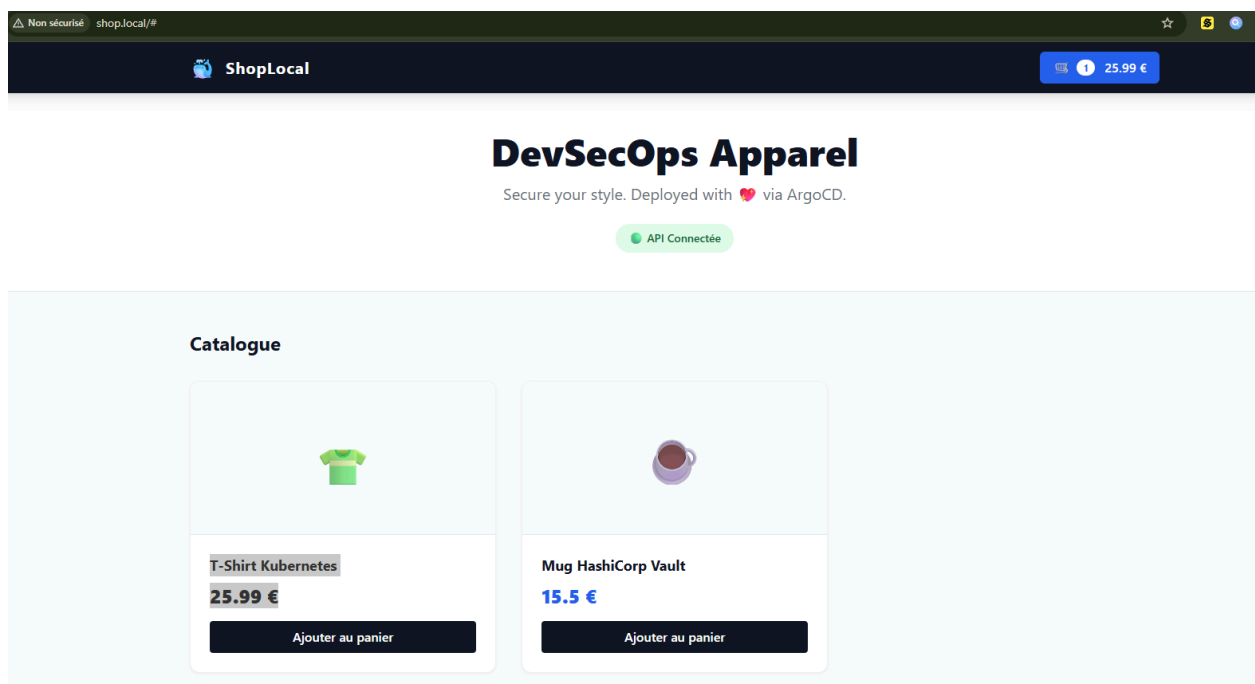
Au moment de démarrer, le moteur Postgres a absolument besoin d'écrire des fichiers temporaires dans d'autres dossiers du système, notamment :

- /var/run/postgresql (pour stocker son fichier socket .s.PGSQL.5432 et son PID).
- /tmp (pour certains calculs temporaires).
-

Si on met tout le disque en lecture seule bêtement, Postgres va crasher au démarrage en disant "Permission denied".

Merci pour votre lecture

Le frontend fut mis à jour afin d'être plus beau visuellement:



Le frontend a été mis à jour pour être davantage plus beau visuellement.

